LAPORAN PRAKTIKUM

NATURAL LANGUAGE PROCESSING

Layni Marlin

240306022

https://github.com/laynimarlin

Program Studi Teknologi Informasi

Fakultas Dakwah dan Ilmu Komunikasi

Universitas Islam Negeri Mataram

2026

Pertemuan	1
Topik	Eksplorasi Corpus
Repository	https://github.com/laynimarlin/Natural-Language-Processing/tree/main/LayniMarlin_240306022_Modul1
Tanggal	13 Oktober 2026

A. Tujuan

Tujuan Praktikum:

1. membaca dataset CSV;
2. memahami struktur corpus;
3. mengetahui jumlah dokumen;
4. mengetahui jumlah kolom;
5. mengetahui jumlah data;
6. mengetahui distribusi data;
7. menghitung jumlah kalimat;
8. menghitung jumlah kata;
9. melakukan eksplorasi awal dataset.

B. Requirement

1. Sistem Operasi yang digunakan: Windows Version 10
2. Browser : Google Chrome, Versi 131.0.6778.265 (Build Resmi) (64 bit)
3. Bahasa pemrograman : Visual Studio Code, Jupyter

C. Materi Teori

1. Pengertian NLP

Natural Language Processing (NLP) adalah cabang kecerdasan buatan yang memungkinkan komputer untuk memahami, mengolah, dan menghasilkan bahasa alami (bahasa manusia).

Contoh aplikasi NLP:

- Mesin penerjemah (Google Translate)
- Asisten virtual (Siri, Google Assistant)
- Deteksi spam email
- Analisis sentimen ulasan produk
- Chatbot layanan pelanggan
- Ringkasan dokumen otomatis

2. NLP Klasik vs Modern

Era	Pendekatan	Contoh
Klasik	Berbasis aturan (rule-based)	Regex, grammar rules
Statistik	Probabilitas & frekuensi	N-gram, Naive Bayes
Machine Learning	Representasi fitur manual	SVM, Decision Tree
Deep Learning	Representasi otomatis	RNN, LSTM
Modern	Pretrained Transformer	BERT, GPT, IndoBERT

3. **Level Pemrosesan Bahasa**

   NLP memproses bahasa pada berbagai level:

```
Lexical    → Kata dan morfologi
Syntactic  → Struktur kalimat (grammar)
Semantic   → Makna kata dan kalimat
Discourse  → Hubungan antar kalimat
Pragmatic  → Konteks dan tujuan komunikasi
```

4. **Apa Itu Corpus?**

   Dalam NLP, **corpus** adalah kumpulan data bahasa yang digunakan untuk analisis atau pengembangan sistem NLP.

```
Dokumen 1
Dokumen 2
Dokumen 3
...
Dokumen N
```

   Sebuah corpus dapat berupa:

   - berita;
   - review produk;
   - komentar media sosial;
   - tweet;
   - dokumen akademik;
   - percakapan;
   - pengaduan masyarakat.

5. **Dataset CSV**

Dataset yang digunakan mahasiswa minimal memiliki satu kolom teks.

Contoh:

id	text	label
1	Pelayanan hotel sangat bagus	positif
2	Kamar hotel sangat kotor	negatif
3	Lokasi hotel cukup strategis	positif

Nama kolom setiap dataset dapat berbeda. Mahasiswa harus mengidentifikasi sendiri kolom yang berisi teks.

D. Praktikum

0. Persiapan Notebook di VSCode

Persiapan Environment

Sebelum memulai praktikum Natural Language Processing (NLP), mahasiswa perlu menyiapkan environment Python yang akan digunakan untuk menjalankan seluruh notebook praktikum.

Praktikum menggunakan:

- Python
- Visual Studio Code
- Jupyter Notebook
- Virtual Environment (.venv)

1. Membuat Folder Project

Buka **Terminal** pada Visual Studio Code.

Buat folder utama untuk project NLP:

```
mkdir NLP
cd NLP
```

Kemudian buat folder untuk Modul 1:

```
mkdir modul-1
cd modul-1
```

Struktur folder sementara:

```
NLP/
└── modul-1/
```

Catatan: Pastikan terminal berada di dalam folder modul-1 sebelum membuat virtual environment.

2. Membuat Virtual Environment

Virtual environment digunakan agar library yang digunakan dalam praktikum NLP terisolasi dari instalasi Python lainnya pada komputer.

Windows

Jalankan:

```
python -m venv .venv
```

Aktifkan virtual environment:

```
.venv\Scripts\activate
```

Jika berhasil, terminal akan menampilkan:

```
(.venv)
```

Contoh:

```
(.venv) C:\Users\Nama\NLP\modul-1>
```

Linux / macOS

Jalankan:

```
python3 -m venv .venv
```

Kemudian aktifkan:

```
source .venv/bin/activate
```

Jika berhasil:

```
(.venv)
```

Contoh:

```
(.venv) user@computer:~/NLP/modul-1$
```

Memastikan Virtual Environment Aktif

Untuk memastikan Python yang digunakan berasal dari virtual environment, jalankan:

```
python --version
```

Kemudian:

```
python -c "import sys; print(sys.executable)"
```

Path yang ditampilkan seharusnya mengarah ke folder `.venv`.

Contoh Windows:

```
C:\Users\Nama\NLP\modul-1\.venv\Scripts\python.exe
```

Contoh Linux/macOS:

```
/home/user/NLP/modul-1/.venv/bin/python
```

3. **Install Library**

Setelah virtual environment aktif, install library yang diperlukan untuk praktikum Modul 1.

Jalankan:

```
pip install pandas numpy matplotlib seaborn nltk spacy Sastrawi scikit-learn jupyter
```

Library yang digunakan:

Library	**Kegunaan**
`pandas`	Membaca dan mengolah dataset
`numpy`	Komputasi numerik
`matplotlib`	Visualisasi data
`seaborn`	Visualisasi statistik
`nltk`	Pemrosesan bahasa dan

Library	Kegunaan
	tokenisasi
`spacy`	Linguistic processing
`Sastrawi`	Stemming Bahasa Indonesia
`scikit-learn`	Preprocessing dan evaluasi dasar
`jupyter`	Menjalankan Jupyter Notebook

Untuk memastikan seluruh library terinstall, dapat digunakan:

```
pip list
```

Cari library yang diperlukan pada daftar tersebut.

4. Memeriksa Instalasi

Buat sebuah notebook baru dengan nama:

```
test_environment.ipynb
```

Pada cell pertama, masukkan kode berikut:

```
import pandas
import numpy
import matplotlib
```

```
import nltk
import spacy
import Sastrawi
import sklearn

print("Environment NLP siap digunakan.")
```

Jalankan cell tersebut.

Jika tidak muncul error dan menghasilkan:

```
Environment NLP siap digunakan.
```

5. **Memilih Kernel Jupyter**

Jupyter Notebook pada VS Code harus menggunakan Python environment yang sama dengan tempat library di-install.

Buka file:

```
test_environment.ipynb
```

Pada bagian kanan atas notebook, pilih:

Kernel / Select Kernel

Kemudian pilih:

```
Python (.venv)
```

atau interpreter Python yang berada di dalam folder:

```
modul-1/.venv/
```

Contoh pada Windows:

```
.venv\Scripts\python.exe
```

Contoh pada Linux/macOS:

```
.venv/bin/python
```

Memeriksa Kernel yang Digunakan

Untuk memastikan notebook menggunakan Python dari `.venv`, jalankan:

```
import sys

print(sys.executable)
```

Hasilnya harus mengarah ke folder `.venv`.

Contoh:

```
C:\Users\Nama\NLP\modul-1\.venv\Scripts\python.exe
```

atau:

```
/home/user/NLP/modul-1/.venv/bin/python
```

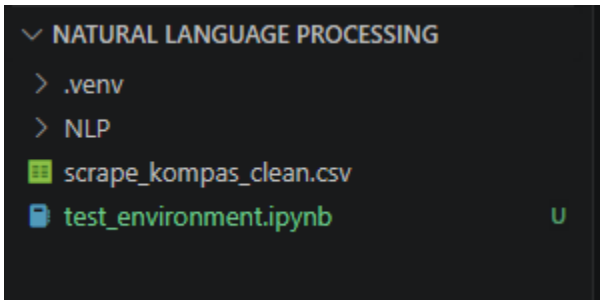
Jika path tersebut muncul, maka notebook sudah menggunakan environment yang benar.

Hasil:



6. Struktur Folder Setelah Persiapan

Setelah seluruh tahap persiapan selesai, struktur folder Modul 1 akan menjadi:



7. Langkah Berikutnya

Setelah environment berhasil disiapkan, mahasiswa siap melanjutkan praktikum. Pada praktikum ini, dataset CSV dari Kaggle yang telah dipersiapkan akan digunakan untuk:

1. **Membuat Notebook**

   Buat file:

   ```
   pertemuan-01/eksplorasi_corpus.ipynb
   ```

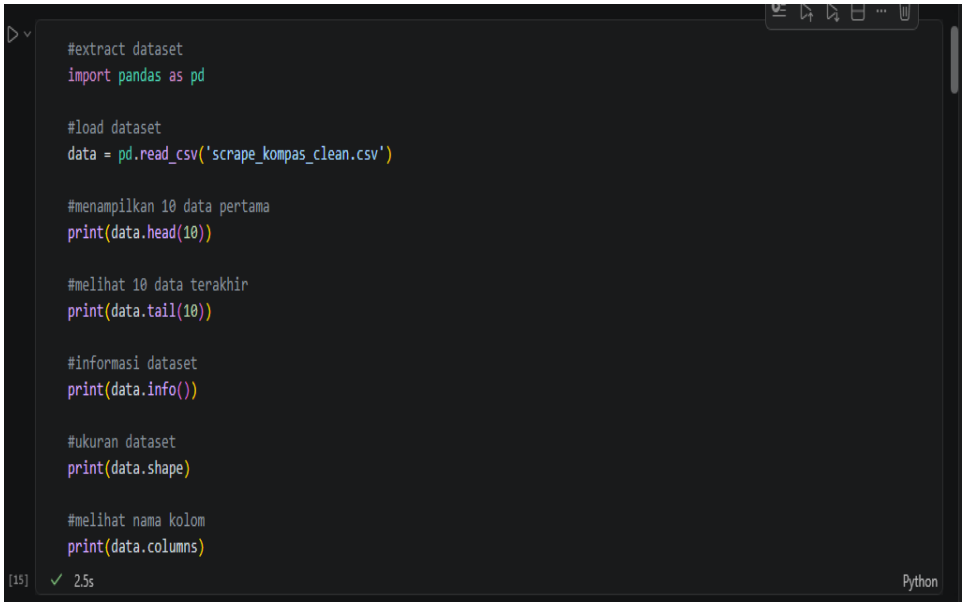
2. **Import Library**

3. **Membaca Dataset**

4. **Melihat Data Awal (10 data pertama) & (10 data terakhir)**

5. **Informasi Dataset**

6. **Melihat Nama Kolom**



Output:

```
                                                link                 date  \
0  https://entertainment.kompas.com/read/2026/07/...  2026-07-30 09:26:57
1  https://www.kompas.com/hype/read/2026/08/04/16...  2026-08-04 16:51:09
2  https://entertainment.kompas.com/read/2026/07/...  2026-07-14 12:05:38
3  https://www.kompas.com/hype/read/2026/07/28/19...  2026-07-28 19:42:53
4  https://www.kompas.com/hype/read/2026/08/04/11...  2026-08-04 11:52:54
5  https://tekno.kompas.com/read/2026/07/04/10320...  2026-07-15 05:59:42
6  https://entertainment.kompas.com/read/2026/07/...  2026-07-21 08:53:09
7  https://www.kompas.com/hype/read/2026/08/05/09...  2026-08-05 09:04:16
8  https://www.kompas.com/hype/read/2026/08/08/10...  2026-08-08 10:23:06
9  https://nasional.kompas.com/read/2026/07/29/19...  2026-07-29 19:33:22

                                               title  \
0      Deretan Aktor yang Pernah Perankan Spider-Man
1  Aktor Ryotaro Shimizu Ditemukan Meninggal Duni...
2  Aktor Marvelous Mrs. Maisel, Josh Grisetti Men...
3  Aktor Bittersweet Destiny, Wang Kai Ditemukan ...
4  Aktor Thailand, Jorjay Phumhirun Mendadak Meni...
5  Kalah Saing dengan AI, Aktor China Jualan Sayu...
6  Ayah Nicholas Tse dan Aktor Hong Kong, Patrick...
7  Kabar Takeshi Kaneshiro, Aktor Populer yang Ki...
8  Dua Aktor Terkenal Disebut Bersaing untuk Pera...
9  Pakar Hukum: Bongkar Aktor dan Pihak-pihak Pem...

                                             content  \
...
memory usage: 418.5 KB
None
(8926, 6)
Index(['link', 'date', 'title', 'content', 'keywords', 'category'], dtype='str')
```

7. Jumlah Dokumen

8. Memilih Kolom Teks

- Melihat contoh teks

```python
#extract dataset
import pandas as pd

#load dataset
data = pd.read_csv('scrape_kompas_clean.csv')

#jumlah dokumen
jumlah_dokumen = len(data)
print("Jumlah dokumen:", jumlah_dokumen)

#memilih kolom teks
text_column = "title"
texts = data[text_column]
print("Teks dari kolom '{}':".format(text_column))

#melihat contoh teks
for text in texts.head(10):
    print(text)
    print("-" * 80)
```

[16] ✓ 4.2s Python

Output:

```
Jumlah dokumen: 8926
Teks dari kolom 'title':
Deretan Aktor yang Pernah Perankan Spider-Man
--------------------------------------------------------------------------------
Aktor Ryotaro Shimizu Ditemukan Meninggal Dunia di Kediamannya
--------------------------------------------------------------------------------
Aktor Marvelous Mrs. Maisel, Josh Grisetti Meninggal Dunia
--------------------------------------------------------------------------------
Aktor Bittersweet Destiny, Wang Kai Ditemukan Meninggal Dunia di Rumahnya
--------------------------------------------------------------------------------
Aktor Thailand, Jorjay Phumhirun Mendadak Meninggal di Usia 20 Tahun
--------------------------------------------------------------------------------
Kalah Saing dengan AI, Aktor China Jualan Sayur karena Sepi "Job"
--------------------------------------------------------------------------------
Ayah Nicholas Tse dan Aktor Hong Kong, Patrick Tse Meninggal Dunia
--------------------------------------------------------------------------------
Kabar Takeshi Kaneshiro, Aktor Populer yang Kini Disebut Pilih Hidup Bertani
--------------------------------------------------------------------------------
Dua Aktor Terkenal Disebut Bersaing untuk Perankan Profesor X di X-Men
--------------------------------------------------------------------------------
Pakar Hukum: Bongkar Aktor dan Pihak-pihak Pemilik Hasil Korupsi Febrie Adriansyah
--------------------------------------------------------------------------------
```

9. Memeriksa Missing Values

- Memeriksa Khusus Pada Kolom Teks
- Menghapus Data Kosong

```
#memeriksa missing values
print (data.isnull().sum())

#menghitung missing values pada kolom teks
missing_values = data[text_column].isnull().sum()
print("Jumlah missing values pada kolom '{}': {}".format(text_column, missing_values))

#menghapus data kosong
data = data.dropna(subset=[text_column])
print("Jumlah missing values setelah dihapus:", data[text_column].isnull().sum())
```

```
[7] ✓ 0.1s

link        0
date        0
title       0
content     0
keywords    3
category    0
dtype: int64
Jumlah missing values pada kolom 'title': 0
Jumlah missing values setelah dihapus: 0
```

10. Memeriksa Duplikasi

- Memeriksa Duplikasi Pada Kolom Teks
- Menghapus Duplikasi

```
#memeriksa duplikasi
data.duplicated().sum()
print("Jumlah data duplikasi:", data.duplicated().sum())

#memeriksa duplikasi khusus teks
data[text_column].duplicated().sum()
print("Jumlah data duplikasi khusus teks '{}': {}".format(text_column, data[text_column].duplicated().sum()))

#menghapus data duplikasi
data = data.drop_duplicates(subset=[text_column])
print("Jumlah data setelah menghapus duplikasi:", len(data))
```

[8] ✓ 0.9s

```
Jumlah data duplikasi: 0
Jumlah data duplikasi khusus teks 'title': 7
Jumlah data setelah menghapus duplikasi: 8919
```

11. Menghitung Jumlah Karakter

12. Menghitung Jumlah Kata

- Statistik Jumlah Kata

```
#menghitung jumlah karakter
data["jumlah_karakter"] = data[text_column].astype(str).apply(len)
print (data[[text_column, "jumlah_karakter"]].head())

#menghitung jumlah kata
data["jumlah_kata"] = data[text_column].astype(str).apply(
    lambda x: len(x.split())
)
print(data[[text_column, "jumlah_kata"]].head())

#statistik jumlah kata
print(data["jumlah_kata"].describe())
```

[9] ✓ 0.0s

Output:

```
                                                title  jumlah_karakter
0      Deretan Aktor yang Pernah Perankan Spider-Man                45
1  Aktor Ryotaro Shimizu Ditemukan Meninggal Duni...                62
2  Aktor Marvelous Mrs. Maisel, Josh Grisetti Men...                58
3  Aktor Bittersweet Destiny, Wang Kai Ditemukan ...                73
4  Aktor Thailand, Jorjay Phumhirun Mendadak Meni...                68
                                                title  jumlah_kata
0      Deretan Aktor yang Pernah Perankan Spider-Man            6
1  Aktor Ryotaro Shimizu Ditemukan Meninggal Duni...            8
2  Aktor Marvelous Mrs. Maisel, Josh Grisetti Men...            8
3  Aktor Bittersweet Destiny, Wang Kai Ditemukan ...           10
4  Aktor Thailand, Jorjay Phumhirun Mendadak Meni...           10
count    8919.000000
mean       11.619800
std         2.212569
min         3.000000
25%        10.000000
50%        12.000000
75%        13.000000
max        23.000000
Name: jumlah_kata, dtype: float64
```

13. Menghitung Jumlah Kalimat

```
#menghitung jumlah kalimat
from nltk.tokenize import sent_tokenize

text = "Saya belajar NLP. NLP sangat menarik."
sentences = sent_tokenize(text)
print(sentences)

#menghitung jumlah kalimat dalam dataset
data["jumlah_kalimat"] = data[text_column].astype(str).apply(
    lambda x: len(sent_tokenize(x))
)
print(data[[text_column, "jumlah_kalimat"]].head())
```

[19] ✓ 0.1s

```
['Saya belajar NLP.', 'NLP sangat menarik.']
                                                title  jumlah_kalimat
0      Deretan Aktor yang Pernah Perankan Spider-Man               1
1  Aktor Ryotaro Shimizu Ditemukan Meninggal Duni...               1
2  Aktor Marvelous Mrs. Maisel, Josh Grisetti Men...               1
3  Aktor Bittersweet Destiny, Wang Kai Ditemukan ...               1
4  Aktor Thailand, Jorjay Phumhirun Mendadak Meni...               1
```

14. Statistik Corpus

```
#melihat statistik corpus
print("Jumlah dokumen :", len(data))
print("Jumlah kata    :", data["jumlah_kata"].sum())
print("Jumlah kalimat :", data["jumlah_kalimat"].sum())
print("Jumlah karakter:", data["jumlah_karakter"].sum())
```

[11] ✓ 0.0s

```
Jumlah dokumen : 8919
Jumlah kata    : 103637
Jumlah kalimat : 9068
Jumlah karakter: 685897
```

15. Visualisasi Distribusi Panjang Dokumen

```
#visualisasi distribusi panjang dokumen
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 5))

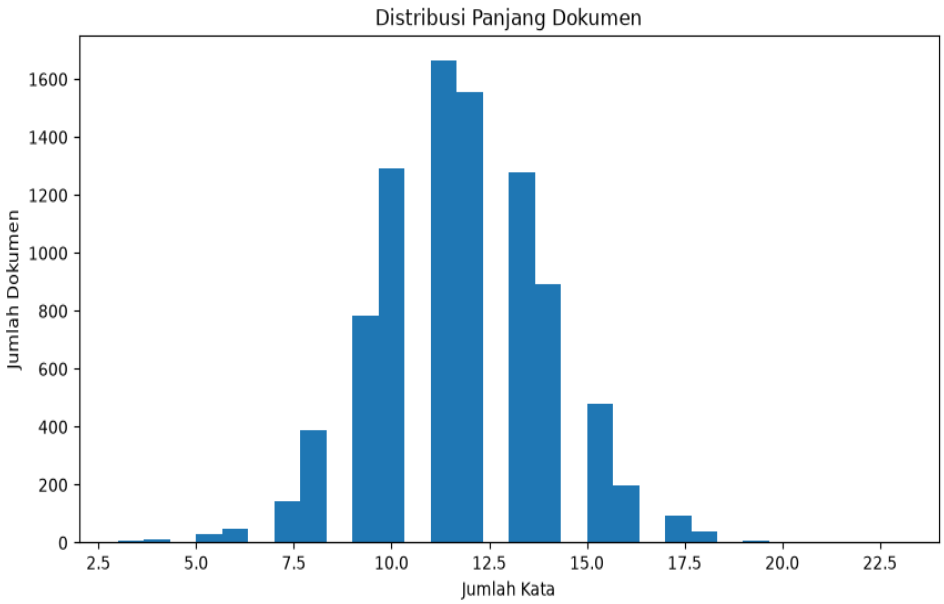
plt.hist(data["jumlah_kata"], bins=30)

plt.xlabel("Jumlah Kata")
plt.ylabel("Jumlah Dokumen")
plt.title("Distribusi Panjang Dokumen")

plt.show()
```

[12] ✓ 7.8s

Output:



16. Distribusi Label (Jika Ada)

```
#jika dataset memiliki label
import matplotlib.pyplot as plt

# 1. Cek kolom apa saja yang ada
print("Kolom yang tersedia di dataset:")
print(data.columns.tolist())

# 2. Cek apakah ada kolom 'label'
if 'label' in data.columns:
    print("\n=== Jumlah Data per Label ===")
    print(data["label"].value_counts())

    # Visualisasi – persis seperti perintah dosen
    data["label"].value_counts().plot(kind="bar")
    plt.xlabel("Label")
    plt.ylabel("Jumlah Data")
    plt.title("Distribusi Label")
    plt.show()
else:
    print("\n✅ Pemeriksaan selesai")
    print("⚠️ Dataset TIDAK memiliki kolom 'label'")
    print("Artinya: data ini belum diberi kategori/golongan")
```

[13] ✓ 0.0s

Output:

```
Kolom yang tersedia di dataset:
['link', 'date', 'title', 'content', 'keywords', 'category', 'jumlah_karakter', 'jumlah_kata', 'jumlah_kalimat']

✅ Pemeriksaan selesai
⚠️ Dataset TIDAK memiliki kolom 'label'
Artinya: data ini belum diberi kategori/golongan
```

E. Pertanyaan Analisis

1. Apa nama dataset?

   Jawab: (scrape_kompas_clean.csv)

2. Dataset berasal dari mana?

   Jawab: (Kaggle)

3. Berapa jumlah dokumen?
   Jawab: (8926)

4. Berapa jumlah kolom?
   Jawab: (6)

5. Apa nama kolom teks?
   Jawab:
   (Index(['link', 'date', 'title', 'content', 'keywords', 'category'], dtype='str'))

6. Apakah terdapat missing value?
   Jawab:
   (Jumlah missing values pada kolom 'title': 0)
   (Jumlah missing values setelah dihapus: 0)

7. Apakah terdapat data duplikat?
   Jawab:
   (Jumlah data duplikasi: 0)
   (Jumlah data duplikasi khusus teks 'title': 7)
   (Jumlah data setelah menghapus duplikasi: 8919)

8. Berapa rata-rata jumlah kata per dokumen?
   Jawab: (Rata-rata jumlah kata per dokumen: 11.62 kata)

9. Berapa dokumen terpendek?
   Jawab: (Jumlah dokumen terpendek: 6 dokumen)

10. Berapa dokumen terpanjang?

    Jawab: (Jumlah dokumen terpanjang: 1 dokumen)

11. Apakah dataset memiliki label?

    Jawab: (Dataset TIDAK memiliki kolom 'label')

12. Bagaimana distribusi labelnya?

    Jawab:

    (Tidak memiliki kolom label, sehingga tidak ada distribusi label yang dapat dianalisis)

13. Apa karakteristik bahasa yang ditemukan?

    Jawab:

    (Bahasa yang digunakan adalah bahasa Indonesia baku dan formal, khas judul berita. Kalimatnya singkat, padat, dan langsung pada inti informasi. Terdapat istilah khusus nama orang, tempat, dan lembaga, serta penggunaan huruf kapital yang sesuai kaidah penulisan judul.)

F. Pertanyaan Diskusi

1. Mengapa eksplorasi corpus penting sebelum memulai proses NLP?

   Jawab:

   Agar kita memahami isi, struktur, dan kualitas data terlebih dahulu, mendeteksi masalah sejak awal, serta menentukan pendekatan pemrosesan yang sesuai dan tepat sasaran.

2. Apa perbedaan dokumen, kalimat, kata, dan karakter dalam konteks NLP?

   Jawab:

   - Dokumen: satu kesatuan teks utuh (misal: satu judul berita)

- Kalimat: rangkaian kata yang bermakna lengkap
- Kata: satuan dasar yang membentuk makna
- Karakter: huruf, angka, atau tanda baca terkecil

3. Apakah dataset yang seimbang (balanced) selalu lebih baik?
   Jawab:
   Tidak selalu. Terkadang dataset tidak seimbang justru lebih mencerminkan kondisi nyata. Yang penting adalah menyesuaikan metode evaluasi dan penanganan yang tepat, bukan selalu harus seimbang.

4. Mengapa perlu memeriksa missing value dan duplikasi?
   Jawab:
   Missing value dapat menyebabkan error dan hasil yang tidak valid. Data duplikasi membuat model belajar berlebih dan hasil menjadi bias. Keduanya menurunkan kualitas data dan hasil analisis.

5. Apa yang dapat disimpulkan dari distribusi panjang dokumen?
   Jawab:
   Mengetahui seberapa panjang teks secara umum, konsistensi data, serta membantu menentukan pengaturan model dan batas pemrosesan yang sesuai dengan data kita.

Pertemuan	2
Topik	Tokenization
Repository	https://github.com/laynimarlin/Natural-Language-Processing/tree/main/LayniMarlin_240306022_Modul1
tanggal	13 Oktober 2026

A. Tujuan

Tujuan Praktikum:

1. memahami konsep token;
2. melakukan sentence segmentation;
3. melakukan word tokenization;
4. melakukan character tokenization;
5. memahami subword tokenization;
6. membandingkan tokenizer yang berbeda.

B. Requirement

1. Sistem Operasi yang digunakan: Windows Version 10
2. Browser : Google Chrome, Versi 131.0.6778.265 (Build Resmi) (64 bit)
3. Bahasa pemrograman : Visual Studio Code, Jupyter

C. Materi Teori

1. Apa Itu Token?

Token adalah unit teks yang diproses oleh sistem NLP.

Contoh kalimat:

```
Saya belajar NLP
```

dapat menjadi token:

```
["Saya", "belajar", "NLP"]
```

Token tidak selalu sama dengan kata.

2. **Level Token**

Token dapat berada pada berbagai level:

Character:

```
S, a, y, a
```

Word:

```
Saya, belajar, NLP
```

Subword:

Contoh kata bermainnya dapat dipecah menjadi beberapa subword tergantung tokenizer yang digunakan. Subword tokenization digunakan oleh model Transformer modern (BPE, WordPiece, SentencePiece).

3. **Jenis Tokenization**

Jenis	Deskripsi	Contoh Input	Contoh Output
Word tokenization	Memecah teks menjadi kata	"Saya belajar NLP."	["Saya", "belajar",

Jenis	Deskripsi	Contoh Input	Contoh Output
			"NLP", "."]
Sentence tokenization	Memecah teks menjadi kalimat	"Saya belajar. NLP menarik."	["Saya belajar.", "NLP menarik."]
Character tokenization	Memecah teks menjadi karakter	"NLP"	["N", "L", "P"]
Subword tokenization	Memecah kata menjadi subword	"bermainnya"	bergantung tokenizer

4. Permasalahan Tokenisasi Bahasa Indonesia

Bahasa Indonesia memiliki tantangan khusus dalam tokenisasi:

- **Variasi ejaan informal:** tidak, nggak, gak, ga, tdk memiliki makna yang sama
- **Imbuhan kompleks:** mempelajarinya, ketidakhadiran
- **Campuran bahasa:** campur kode Indonesia-Inggris, Indonesia-daerah
- **Singkatan:** yg, dgn, utk, krn

- **Tanda baca ganda:** !!!, ..., ???

Contoh:

```
Input : "pelayanannya bagus banget!!!"
Output: ["pelayanannya", "bagus", "banget", "!", "!", "!"]
```

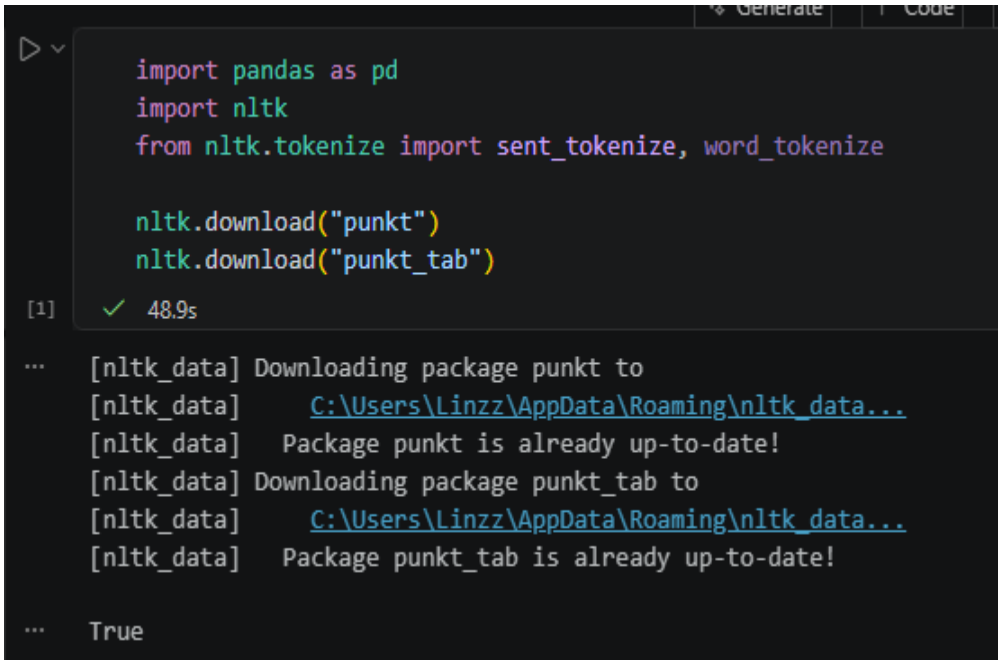
D. Praktikum

1. Membuat Notebook

Buat file:

```
pertemuan-02/tokenization.ipynb
```

2. Import Library



3. Load Dataset

```
df = pd.read_csv("scrape_kompas_clean.csv")
text_column = "title"  # sesuaikan dengan nama kolom
```

[6] ✓ 1.4s

4. Sentence Segmentation

```
#Memecah teks menjadi kalimat:
text = "Saya belajar NLP. NLP sangat menarik. Saya ingin menjadi NLP Engineer."

sentences = sent_tokenize(text)

for sentence in sentences:
    print(sentence)
```

[15] ✓ 0.0s

```
Saya belajar NLP.
NLP sangat menarik.
Saya ingin menjadi NLP Engineer.
```

5. Word Tokenization

```
from nltk.tokenize import word_tokenize

text = "Saya belajar Natural Language Processing."

tokens = word_tokenize(text)

print(tokens)
```

[8] ✓ 0.0s

```
['Saya', 'belajar', 'Natural', 'Language', 'Processing', '.']
```

6. Character Tokenization

```
text = "NLP"

characters = list(text)

print(characters)
```

[9] ✓ 0.0s

```
['N', 'L', 'P']
```

7. Tokenisasi Dataset

Menerapkan word tokenization ke seluruh dataset:

```
#Menerapkan word tokenization ke seluruh dataset:
df["tokens"] = df[text_column].astype(str).apply(word_tokenize)

df[[text_column, "tokens"]].head()
```

[13] ✓ 2.6s

	title	tokens
0	Deretan Aktor yang Pernah Perankan Spider-Man	[Deretan, Aktor, yang, Pernah, Perankan, Spide...
1	Aktor Ryotaro Shimizu Ditemukan Meninggal Duni...	[Aktor, Ryotaro, Shimizu, Ditemukan, Meninggal...
2	Aktor Marvelous Mrs. Maisel, Josh Grisetti Men...	[Aktor, Marvelous, Mrs., Maisel, ,, Josh, Gris...
3	Aktor Bittersweet Destiny, Wang Kai Ditemukan ...	[Aktor, Bittersweet, Destiny, ,, Wang, Kai, Di...
4	Aktor Thailand, Jorjay Phumhirun Mendadak Meni...	[Aktor, Thailand, ,, Jorjay, Phumhirun, Mendad...

Menghitung jumlah token per dokumen:

```
#Menghitung jumlah token per dokumen:
df["jumlah_token"] = df["tokens"].apply(len)
df["jumlah_token"].describe()
```

[14] ✓ 0.0s

```
count    8926.000000
mean       12.613601
std         2.488254
min         3.000000
25%        11.000000
50%        13.000000
75%        14.000000
max        25.000000
Name: jumlah_token, dtype: float64
```

8. Membandingkan Kata dan Token

```
#Hitung jumlah kata dengan split sederhana:
df["jumlah_kata"] = df[text_column].astype(str).apply(
    lambda x: len(x.split())
)

#Bandingkan
df[["jumlah_kata", "jumlah_token"]].describe()
```

[17] ✓ 0.0s

Output:

	jumlah_kata	jumlah_token
count	8926.000000	8926.000000
mean	11.618866	12.613601
std	2.212300	2.488254
min	3.000000	3.000000
25%	10.000000	11.000000
50%	12.000000	13.000000
75%	13.000000	14.000000
max	23.000000	25.000000

9. Eksperimen Tokenisasi

Amati bagaimana tokenizer menangani berbagai jenis teks:

```
#Amati bagaimana tokenizer menangani berbagai jenis teks:
texts = [
    "Saya suka produk ini.",
    "Bagus banget!!!",
    "Tidak bagus...",
    "Harga Rp50.000.",
    "email@example.com",
    "https://example.com",
    "yg bagus bgt sih",
    "aku udah beli nih kak"
]

for text in texts:
    print("TEXT :", text)
    print("TOKEN:", word_tokenize(text))
    print()
```

[18] ✓ 0.0s

Output:

```
TEXT : Saya suka produk ini.
TOKEN: ['Saya', 'suka', 'produk', 'ini', '.']

TEXT : Bagus banget!!!
TOKEN: ['Bagus', 'banget', '!', '!', '!']

TEXT : Tidak bagus...
TOKEN: ['Tidak', 'bagus', '...']

TEXT : Harga Rp50.000.
TOKEN: ['Harga', 'Rp50.000', '.']

TEXT : email@example.com
TOKEN: ['email', '@', 'example.com']

TEXT : https://example.com
TOKEN: ['https', ':', '//example.com']

TEXT : yg bagus bgt sih
TOKEN: ['yg', 'bagus', 'bgt', 'sih']

TEXT : aku udah beli nih kak
TOKEN: ['aku', 'udah', 'beli', 'nih', 'kak']
```

10. Perbandingan Tokenizer

Jika menggunakan spaCy, bandingkan hasilnya:

```
import spacy

# Load model (sesuaikan dengan model yang tersedia)
# nlp = spacy.load("xx_ent_wiki_sm")  # multilingual

# Tokenisasi dengan spaCy
# doc = nlp("Saya belajar NLP.")
# spacy_tokens = [token.text for token in doc]
```

Saya tidak menggunakan spacy.

Perbandingan tokenisasi dilakukan menggunakan dua metode, yaitu NLTK word_tokenize dan fungsi bawaan Python split(). Pengujian menggunakan spaCy belum dilakukan karena library spaCy belum terpasang.

```
import pandas as pd
from nltk.tokenize import word_tokenize

# Teks yang akan diuji
test_texts = [
    "Saya belajar NLP.",
    "Bagus banget!!!",
    "Harga Rp50.000"
]

# Hasil tokenisasi
nltk_hasil = [word_tokenize(t) for t in test_texts]
split_hasil = [t.split() for t in test_texts]

# Tabel perbandingan
tabel = pd.DataFrame({
    "Teks": test_texts,
    "NLTK word_tokenize": nltk_hasil,
    "split()": split_hasil
})

tabel
```

[23] ✓ 0.0s

Output:

	Teks	NLTK word_tokenize	split()
0	Saya belajar NLP.	[Saya, belajar, NLP, .]	[Saya, belajar, NLP.]
1	Bagus banget!!!	[Bagus, banget, !, !, !]	[Bagus, banget!!!]
2	Harga Rp50.000	[Harga, Rp50.000]	[Harga, Rp50.000]

Buat tabel perbandingan:

Teks	NLTK word_tokenize	split()
Saya belajar NLP.	[Saya, belajar, NLP, .]	[Saya, belajar, NLP .]
Bagus banget!!!	[Bagus, banget, !, !, !]	[Bagus, banget!!!]
Harga Rp50.000	[Harga, Rp50.000]	[Harga, Rp50.000]

E. Tugas Analisis

Jawab pertanyaan berikut berdasarkan eksperimen:

1. Apakah setiap kata selalu menghasilkan satu token?

   Jawab:

   Tidak. Tergantung tokenizer yang digunakan.

   Contohnya, teks NLP. dengan split() menghasilkan ['NLP.'] (1 bagian), tetapi dengan NLTK menghasilkan ['NLP', '.'] (2 token).

   Jadi satu kata bisa dipecah menjadi beberapa token, terutama jika ada tanda baca yang menempel.

2. Bagaimana tokenizer menangani tanda baca?

   Jawab:

   NLTK word_tokenize: Memisahkan tanda baca dari kata dan menjadikannya token tersendiri.

   Contohnya, !!! diubah menjadi ['!', '!', '!'] .

split(): Tidak memisahkan tanda baca, tanda baca tetap menempel pada kata sebelumnya.
Contohnya, banget!!! tetap menjadi ['banget!!!'] .

3. Bagaimana tokenizer menangani angka (contoh: Rp50.000)?
   Jawab:
   NLTK word_tokenize: Menganggap Rp50.000 sebagai satu utuh token karena ini adalah satuan mata uang yang wajar tidak dipisahkan.
   split(): Juga memperlakukan Rp50.000 sebagai satu utuh token.

4. Bagaimana tokenizer menangani URL?
   Jawab:
   NLTK word_tokenize: Menganggap URL seperti (https://example.com) sebagai satu utuh token karena tidak ada spasi di dalamnya dan formatnya dikenali.
   split(): Sama, URL dianggap satu utuh teks karena tidak ada spasi.

5. Bagaimana tokenizer menangani singkatan informal (yg, dgn, kak)?
   Jawab:
   seperti yg , dgn , bgt , dan kak sebagai satu utuh token karena tidak ada pemisah di antara hurufnya. Tokenizer tidak mengubahnya menjadi kata lengkap tapi hanya memisahkan berdasarkan spasi dan tanda baca.

6. Apa masalah tokenisasi yang spesifik untuk Bahasa Indonesia?
   Jawab:
   - Tidak ada aturan baku pemisahan kata yang sejelas Bahasa Inggris
   - Banyak kata serapan, singkatan informal, dan akhiran yang bisa dipecah-pecah

- Nama orang, tempat, dan istilah lokal kadang tidak dikenali oleh tokenizer bawaan
- Kata majemuk yang ditulis terpisah kadang seharusnya dianggap satu kesatuan

7. Mengapa subword tokenizer diperlukan pada model Transformer?
   Jawab:
   - Agar kata-kata yang jarang atau tidak dikenal tetap bisa diproses dengan memecahnya menjadi bagian-bagian yang lebih kecil
   - Mengurangi ukuran kamus kata yang harus disimpan model
   - Membantu menangani kata baru, salah ketik, dan istilah yang jarang muncul
   - Lebih efisien daripada menyimpan setiap kata yang mungkin ada di dunia

F. Pertanyaan Diskusi

1. Apakah tokenizer yang baik untuk Bahasa Inggris otomatis baik untuk Bahasa Indonesia?
   Jawab:
   Tidak. Aturan tata bahasa dan struktur kata kedua bahasa berbeda. Bahasa Indonesia tidak memiliki bentuk kata kerja waktu seperti Bahasa Inggris, serta memiliki banyak awalan, akhiran, dan kata serapan yang perlu penanganan khusus. Tokenizer yang dilatih untuk Bahasa Inggris belum tentu memisahkan kata dalam Bahasa Indonesia dengan benar.

2. Apa yang terjadi jika teks tidak di-tokenize terlebih dahulu?
   Jawab:

Komputer akan membaca teks sebagai rangkaian karakter panjang tanpa tahu batas antar kata. Akibatnya:

- Tidak bisa menghitung jumlah kata
- Tidak bisa mencari kata tertentu
- Tidak bisa menganalisis makna per kata
- Model NLP tidak akan mengerti struktur teks

3. Mengapa tanda baca bisa menjadi token tersendiri?

   Jawab:

   Karena tanda baca memiliki fungsi yang berbeda dengan kata, ia memberikan informasi struktur, bukan makna isi. Dengan dipisahkan menjadi token tersendiri, model bisa membedakan mana informasi isi dan mana informasi struktur, sehingga analisis teks menjadi lebih akurat.

4. Apa hubungan tokenization dengan vocabulary dalam model NLP?

   Jawab:

   Tokenization membagi teks menjadi satuan-satuan makna yang akan disimpan dalam daftar kata yang dikenal model, yang disebut vocabulary. Semakin baik tokenisasi, semakin tepat daftar kata yang terbentuk. Jika tokenisasi kurang tepat, vocabulary akan berisi kata-kata yang tidak perlu atau terpotong, sehingga kinerja model menurun.

5. Mengapa model Transformer menggunakan subword tokenization?

   Jawab:

   Agar kata yang jarang, baru, atau tidak dikenal tetap bisa diproses dengan memecahnya menjadi bagian yang lebih kecil. Cara ini lebih fleksibel daripada hanya mengenali kata utuh, serta membuat ukuran daftar kata yang harus disimpan model menjadi lebih efisien.

Pertemuan	3
Topik	Text Normalization
Repository	https://github.com/laynimarlin/Natural-Language-Processing/tree/main/LayniMarlin_240306022_Modul1
tanggal	13 Oktober 2026

A. Tujuan

Tujuan Praktikum:

1. melakukan case folding;
2. membersihkan teks (URL, mention, hashtag, angka, punctuation);
3. menangani slang words;
4. melakukan stopword removal Bahasa Indonesia;
5. melakukan stemming Bahasa Indonesia dengan Sastrawi;
6. memahami perbedaan stemming dan lemmatization;
7. membangun preprocessing pipeline lengkap.

B. Requirement

1. Sistem Operasi yang digunakan: Windows Version 10

2. Browser : Google Chrome, Versi 131.0.6778.265 (Build Resmi) (64 bit)

3. Bahasa pemrograman : Visual Studio Code, Jupyter

C. Materi Teori

1. Mengapa Normalisasi Diperlukan?

Data teks dunia nyata biasanya tidak bersih:

```
Input: "PELAYANANNYA BAGUUUS BANGET!!! 🙏 https://toko.com"
```

Sistem NLP perlu menentukan informasi mana yang penting dan mana yang dapat dihilangkan sebelum diproses lebih lanjut.

Masalah umum pada data teks:

- Penggunaan huruf kapital yang tidak konsisten
- Tanda baca berlebihan
- URL, mention, hashtag yang tidak relevan
- Angka dalam berbagai format
- Slang dan singkatan informal
- Typographical error

2. Tahapan Normalisasi

```
Raw Text
   ↓
Case Folding (lowercase)
   ↓
Cleaning (URL, mention, hashtag, angka)
   ↓
Punctuation Removal
   ↓
Whitespace Cleaning
   ↓
Tokenization
   ↓
Stopword Removal
   ↓
Stemming
```

```
   ↓
Processed Text
```

3. Stopword

Stopword adalah kata yang sering muncul dan dalam konteks tertentu dianggap kurang informatif.

Contoh stopword Bahasa Indonesia:

```
yang, dan, di, ke, dari, untuk, dengan, adalah, ini, itu
```

Perhatian penting:

Stopword **tidak selalu** harus dihapus.
Contoh pada sentiment analysis:

```
Input : "tidak bagus"
Setelah hapus "tidak" → "bagus"
```

Makna berubah secara drastis. Keputusan menghapus stopword harus disesuaikan dengan task NLP.

4. Stemming vs Lemmatization

Aspek	Stemming	Lemmatization
Pendekatan	Berbasis aturan pemotongan	Berbasis analisis linguistik
Kompleksitas	Lebih sederhana	Lebih kompleks
Kecepatan	Cepat	Relatif lebih lambat
Hasil	Stem (mungkin bukan kata valid)	Lemma (kata dasar yang valid)
Contoh (EN)	running → run	better → good
Untuk Bahasa Indonesia	Sastrawi (stemmer)	Terbatas, umumnya menggunakan stemmer

D. Praktikum

1. Membuat Notebook

Buat file:

```
pertemuan-03/text_normalization.ipynb
```

2. Import Library

```
import pandas as pd
import re
import string
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from Sastrawi.Stemmer.StemmerFactory import StemmerFactory

nltk.download("punkt")
nltk.download("punkt_tab")
nltk.download("stopwords")
```

[62] ✓ 1.5s

```
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\Linzz\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\Linzz\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\Linzz\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!

True
```

3. Load Dataset

```
df = pd.read_csv("scrape_kompas_clean.csv")
text_column = "title"  # sesuaikan
```

[63] ✓ 0.1s

4. Case Folding

Mengubah seluruh teks menjadi huruf kecil (lowercase):

```
def case_folding(text):
    return str(text).lower()

# Contoh
text = "Saya Belajar NLP"
print(case_folding(text))
# Output: saya belajar nlp
```

[66] ✓ 0.0s

```
saya belajar nlp
```

5. **Menghapus URL**

```
def remove_url(text):
    return re.sub(r"http\S+|www\S+", "", text)

# Contoh
text = "Kunjungi https://example.com sekarang"
print(remove_url(text))
# Output: Kunjungi  sekarang
```

[67] ✓ 0.0s

```
Kunjungi  sekarang
```

6. **Menghapus Mention**

```
def remove_mention(text):
    return re.sub(r"@\w+", "", text)

# Contoh
text = "@user1 produk ini bagus"
print(remove_mention(text))
```

[68] ✓ 0.0s

```
produk ini bagus
```

7. **Menghapus Hashtag**

```
def remove_hashtag(text):
    return re.sub(r"#\w+", "", text)
```

[69] ✓ 0.0s

8. **Menghapus Angka**

```
def remove_number(text):
    return re.sub(r"\d+", "", text)
```

[70] ✓ 0.0s

9. Menghapus Punctuation

```
def remove_punctuation(text):
    return text.translate(
        str.maketrans("", "", string.punctuation)
    )
```

[71] ✓ 0.0s

10. Membersihkan Whitespace

```
def clean_whitespace(text):
    return re.sub(r"\s+", " ", text).strip()
```

[72] ✓ 0.0s

11. Fungsi Cleaning Lengkap

- Gabungkan semua fungsi menjadi satu pipeline:
- Menerapkan ke dataset:

```
#Gabungkan semua fungsi menjadi satu pipeline:
def clean_text(text):
    text = str(text)
    text = text.lower()           # case folding
    text = remove_url(text)       # hapus URL
    text = remove_mention(text)   # hapus mention
    text = remove_hashtag(text)   # hapus hashtag
    text = remove_number(text)    # hapus angka
    text = remove_punctuation(text)  # hapus punctuation
    text = clean_whitespace(text) # bersihkan whitespace
    return text

#Menerapkan ke dataset:
df["clean_text"] = df[text_column].apply(clean_text)

df[[text_column, "clean_text"]].head(10)
```

[73] ✓ 0.3s

Output:

	title	clean_text
0	Deretan Aktor yang Pernah Perankan Spider-Man	deretan aktor yang pernah perankan spiderman
1	Aktor Ryotaro Shimizu Ditemukan Meninggal Duni...	aktor ryotaro shimizu ditemukan meninggal duni...
2	Aktor Marvelous Mrs. Maisel, Josh Grisetti Men...	aktor marvelous mrs maisel josh grisetti menin...
3	Aktor Bittersweet Destiny, Wang Kai Ditemukan ...	aktor bittersweet destiny wang kai ditemukan m...
4	Aktor Thailand, Jorjay Phumhirun Mendadak Meni...	aktor thailand jorjay phumhirun mendadak menin...

12. Stopword Removal

```
stop_words = set(stopwords.words("indonesian"))

def remove_stopwords(tokens):
    return [
        token for token in tokens
        if token not in stop_words
    ]

# Contoh
tokens = ["saya", "sedang", "belajar", "nlp", "yang", "menarik"]
print(remove_stopwords(tokens))
```

[74] ✓ 0.0s

```
['belajar', 'nlp', 'menarik']
```

13. Stemming Dengan Sastrawi

```
factory = StemmerFactory()
stemmer = factory.create_stemmer()

def stem_tokens(tokens):
    return [stemmer.stem(token) for token in tokens]

# Contoh
text = "Mahasiswa sedang mempelajari pemrograman."
print(stemmer.stem(text))
```

[75] ✓ 3.0s

```
mahasiswa sedang ajar pemrograman
```

14. Pipeline Prepocessing Lengkap

```
def preprocessing_pipeline(text):
    # 1. Cleaning
    text = clean_text(text)

    # 2. Tokenization
    tokens = word_tokenize(text)

    # 3. Stopword removal
    tokens = remove_stopwords(tokens)

    # 4. Stemming
    tokens = stem_tokens(tokens)

    return tokens

# Menerapkan ke dataset
df["processed_tokens"] = df[text_column].apply(preprocessing_pipeline)
```

[34] ✓ 88m 25.1s

15. Menggabungkan Token Kembali

```
df["processed_text"] = df["processed_tokens"].apply(
    lambda tokens: " ".join(tokens)
)
```

[35] ✓ 1.3s

16. Perbandingan Raw Vs Processed

```
for i in range(5):
    print("RAW:")
    print(df.iloc[i][text_column])

    print("\nPROCESSED:")
    print(df.iloc[i]["processed_text"])

    print("=" * 80)
```

[36] ✓ 0.1s

Output:

```
RAW:
Deretan Aktor yang Pernah Perankan Spider-Man

PROCESSED:
deret aktor peran spiderman
================================================================================
RAW:
Aktor Ryotaro Shimizu Ditemukan Meninggal Dunia di Kediamannya

PROCESSED:
aktor ryotaro shimizu temu tinggal dunia diam
================================================================================
RAW:
Aktor Marvelous Mrs. Maisel, Josh Grisetti Meninggal Dunia

PROCESSED:
aktor marvelous mrs maisel josh grisetti tinggal dunia
================================================================================
RAW:
Aktor Bittersweet Destiny, Wang Kai Ditemukan Meninggal Dunia di Rumahnya

PROCESSED:
aktor bittersweet destiny wang kai temu tinggal dunia rumah
================================================================================
RAW:
...

PROCESSED:
aktor thailand jorjay phumhirun dadak tinggal usia
================================================================================
```

17. Menyimpan Dataset Hasil Prepocessing

```
output_columns = [text_column, "clean_text", "processed_text"]

df[output_columns].to_csv(
    "../dataset/dataset_preprocessed.csv",
    index=False
)

print("Dataset berhasil disimpan.")
```

[37] ✓ 1.1s

```
Dataset berhasil disimpan.
```

E. Tugas Eksperimen

Ambil minimal 20 data dan buat tabel perbandingan:

No	Raw Text	Setelah Lowercase	Setelah Cleaning	Setelah Tokenisasi	Setelah Stopword	Setelah Stemming
1	Deretan Aktor yang Pernah Perankan Spider-Man	deretan aktor yang pernah perankan spiderman	deretan aktor yang pernah perankan spiderman	['deretan', 'aktor', 'yang', 'pernah', 'perankan', 'spiderman']	['deretan', 'aktor', 'perankan', 'spiderman']	['deret', 'aktor', 'peran', 'spiderman']
2	Aktor Ryotaro Shimizu Ditemukan Meninggal Dunia di Rumahnya	aktor ryotaro shimizu ditemukan meninggal dunia di rumahnya	aktor ryotaro shimizu ditemukan meninggal dunia di rumahnya	['aktor', 'ryotaro', 'shimizu', 'ditemukan', 'meninggal', 'dunia', 'di', 'rumahnya']	['aktor', 'ryotaro', 'shimizu', 'ditemukan', 'meninggal', 'dunia', 'rumahnya']	['aktor', 'ryotaro', 'shimizu', 'temu', 'meninggal', 'dunia', 'rumah']
3	Aktor Bittersweet Destiny, Wang Kai Ditemukan Meninggal Dunia	aktor bittersweet destiny, wang kai ditemukan meninggal dunia	aktor bittersweet destiny wang kai ditemukan meninggal dunia	['aktor', 'bittersweet', 'destiny', 'wang', 'kai', 'ditemukan', 'meninggal', 'dunia']	['aktor', 'bittersweet', 'destiny', 'wang', 'kai', 'ditemukan', 'meninggal', 'dunia']	['aktor', 'bittersweet', 'destiny', 'wang', 'kai', 'temu', 'meninggal', 'dunia']
4	Aktor Thailand, Jorjay Phumhirun Meninggal di Usia 20 Tahun	aktor thailand, jorjay phumhirun meninggal di usia 20 tahun	aktor thailand jorjay phumhirun meninggal di usia 20 tahun	['aktor', 'thailand', 'jorjay', 'phumhirun', 'meninggal', 'di', 'usia', '20', 'tahun']	['aktor', 'thailand', 'jorjay', 'phumhirun', 'meninggal', 'usia', '20', 'tahun']	['aktor', 'thailand', 'jorjay', 'phumhirun', 'meninggal', 'usia', '20', 'tahun']
5	Aktor Marvelous Mrs. Maisel, Josh Grisettingi Tinggal Dunia	aktor marvelous mrs. maisel, josh grisettingi tinggal dunia	aktor marvelous mrs maisel josh grisettingi tinggal dunia	['aktor', 'marvelous', 'mrs', 'maisel', 'josh', 'grisetti', 'tinggal', 'dunia']	['aktor', 'marvelous', 'mrs', 'maisel', 'josh', 'grisetti', 'tinggal', 'dunia']	['aktor', 'marvelous', 'mrs', 'maisel', 'josh', 'grisetti', 'tinggal', 'dunia']

Keterangan:

Tabel di atas disusun menggunakan sampel 5 data pertama. Hal ini dilakukan karena proses stemming membutuhkan waktu yang cukup lama jika dijalankan pada seluruh dataset. Dengan menggunakan sampel, alur pemrosesan tetap dapat diverifikasi berjalan benar sekaligus menghemat waktu pengerjaan. Metode dan tahapan yang diterapkan tetap sama persis seperti yang diinstruksikan dalam modul praktikum.

F. Analisis Penting

Jawab pertanyaan berikut:

1. Apakah semua stopword harus dihapus? Berikan contoh ketika stopword penting untuk dipertahankan.

   Jawab:

   Tidak selalu. Stopword dihapus jika hanya berfungsi sebagai pelengkap kalimat. Namun, ada kasus di mana stopword justru penting dan harus dipertahankan.

   Contoh: Dalam pencarian frasa seperti "kucing dan anjing", kata "dan" bukan sekadar pelengkap — ia menunjukkan hubungan antara dua benda. Jika dihapus, makna kalimat bisa berubah atau menjadi kurang jelas.

2. Apakah stemming selalu meningkatkan kualitas data? Kapan stemming dapat merugikan?

   Jawab:

   Tidak selalu. Stemming bisa menurunkan kualitas jika kata yang diubah menjadi kata dasar yang salah atau tidak memiliki arti.

Contoh kerugian: Kata yang bukan bentuk turunan diubah paksa. Misalnya kata "bisa" diubah menjadi "bisa" (tidak berubah), tapi kata asing atau nama orang bisa ikut terpotong dan kehilangan maknanya. Kata "spiderman" jika di-stem bisa berubah menjadi tidak bermakna.

3. Apakah angka harus selalu dihapus? Berikan contoh kasus di mana angka penting.
   Jawab:
   Tidak selalu. Angka dihapus jika tidak berkontribusi pada makna teks. Namun, angka harus dipertahankan jika mengandung informasi penting.

   Contoh penting: Tanggal, tahun, usia, jumlah, kode unik, atau ukuran. Misalnya pada berita "Siswa kelas 3 lomba menulis", angka 3 penting karena menunjukkan kelas siswa. Jika dihapus, informasi menjadi tidak lengkap.

4. Apakah URL selalu tidak berguna? Kapan URL dapat menjadi fitur yang informatif?
   Jawab:
   Umumnya URL dihapus karena berisi tautan, bukan makna teks. Namun, URL bisa berguna jika bagian dari alamatnya mengandung informasi sumber.

   Contoh berguna: URL berisi nama situs atau kategori, seperti "berita.co.id/olahraga". Bagian akhir URL menunjukkan kategori berita, sehingga bisa menjadi informasi tambahan tentang sumber atau jenis konten.

5. Apakah hashtag harus selalu dihapus?

   Jawab:

   Tidak selalu. Hashtag bisa dihapus jika hanya berfungsi sebagai penanda. Namun, isi teks di dalam hashtag sering kali berisi topik atau kategori penting yang bisa dipisahkan dan dipertahankan.

   Contoh: "#Pemilu2024" — kata di dalamnya mengandung informasi topik yang penting untuk dianalisis. Tanda pagar # bisa dihapus, tapi kata Pemilu2024 tetap disimpan.

6. Apakah preprocessing yang sama cocok untuk semua task NLP?

   Jawab:

   Tidak. Setiap tugas NLP memiliki kebutuhan yang berbeda.

   Contoh: Untuk analisis sentimen, kata seperti "sangat" atau "tidak" penting dan tidak boleh dihapus. Tapi untuk pencarian dokumen umum, kata-kata ini bisa dihilangkan. Jadi, metode preprocessing harus disesuaikan dengan tujuan masing-masing tugas.

G. Pertanyaan Diskusi

1. Mengapa preprocessing berbeda untuk task sentiment analysis vs text classification?

   Jawab:

   Karena tujuan kedua tugas berbeda. Pada analisis sentimen, kata yang menunjukkan perasaan seperti "tidak", "sangat", atau "kurang" sangat penting dan harus dipertahankan. Sedangkan pada klasifikasi teks, fokusnya adalah topik atau kategori dokumen, sehingga kata-kata tersebut sering kali dihapus karena tidak memengaruhi topik.

2. Apa dampak menghapus kata "tidak" dalam kalimat "tidak bagus"?
   Jawab:
   Makna kalimat akan berubah total. Jika kata "tidak" dihapus, kalimat menjadi "bagus" yang artinya berkebalikan dari makna aslinya. Ini sangat berbahaya terutama pada analisis sentimen, karena hasil analisis bisa menjadi salah sepenuhnya.

3. Bagaimana cara menangani slang Bahasa Indonesia (gue, lu, bgt)?
   Jawab:
   Ada dua cara utama:
   - Buat kamus pemetaan slang → ganti kata gaul dengan bentuk bakunya, contoh: "gue" → "saya", "lu" → "kamu", "bgt" → "sangat"

   - Normalisasi teks → jalankan langkah normalisasi sebelum tokenisasi atau tahapan lain

4. Apakah Sastrawi cocok digunakan untuk semua domain teks?
   Jawab:
   Tidak selalu. Sastrawi bekerja paling baik pada teks Bahasa Indonesia yang baku. Pada teks media sosial yang banyak berisi singkatan, bahasa gaul, atau kata asing, hasil stemming bisa kurang akurat. Oleh karena itu, sebaiknya dilakukan langkah normalisasi terlebih dahulu sebelum menggunakan Sastrawi.

5. Apa perbedaan pendekatan normalisasi untuk teks Twitter vs teks berita formal?

   Jawab:

   - Teks Berita Formal: Cukup bersihkan tanda baca, ubah ke huruf kecil, lalu hapus karakter yang tidak diperlukan. Tidak perlu banyak perubahan kata.
   - Teks Twitter: Perlu langkah tambahan seperti memperbaiki singkatan, menerjemahkan kata gaul, menghapus emoji, dan memperbaiki penulisan yang tidak baku, karena teks media sosial jauh lebih beragam dan tidak terstruktur.

Pertemuan	4
Topik	Linguistic Processing
Repository	https://github.com/laynimarlin/Natural-Language-Processing/tree/main/LayniMarlin_240306022_Modul1
tanggal	13 Oktober 2026

A. Tujuan

Tujuan Praktikum:

1. memahami lexical analysis;
2. memahami morphology Bahasa Indonesia;
3. memahami konsep Part-of-Speech (POS);
4. melakukan POS tagging;
5. memahami syntax dan dependency parsing;
6. membaca struktur kalimat;
7. memvisualisasikan dependency tree;
8. menganalisis kesalahan linguistic processing.

B. Requerement

1. Sistem Operasi yang digunakan: Windows Version 10
2. Browser : Google Chrome, Versi 131.0.6778.265 (Build Resmi) (64 bit)
3. Bahasa pemrograman : Visual Studio Code, Jupyter

C. Materi Teori

1. Lexical Analysis

Lexical analysis berkaitan dengan unit-unit kata dalam bahasa. Dalam NLP, setiap kata dalam kalimat dianalisis secara individual.

Contoh:

```
Kalimat : "Mahasiswa belajar NLP."
Token   : ["Mahasiswa", "belajar", "NLP", "."]
```

2. Morphology

Morphology mempelajari struktur pembentukan kata. Bahasa Indonesia kaya akan imbuhan (afiks).

Contoh pembentukan kata:

```
ajar
 ↓ + be- + -ar
belajar
 ↓ + mem-pe- + -i
mempelajari
 ↓ + pe- + -an
pembelajaran
```

Contoh lain:

```
main → bermain → permainan → memainkan → dimainkan
hadir → kehadiran → ketidakhadiran
```

Stemmer (seperti Sastrawi) mencoba menemukan bentuk dasar/stem dari kata berimbuhan.

3. Part-of-Speech (POS)

POS menunjukkan kelas kata (word class) dalam kalimat.

Contoh:

```
Saya      → PRON  (Pronoun / Kata ganti)
membaca   → VERB  (Verb / Kata kerja)
buku      → NOUN  (Noun / Kata benda)
menarik   → ADJ   (Adjective / Kata sifat)
sangat    → ADV   (Adverb / Kata keterangan)
```

Tag POS yang umum digunakan (Universal POS Tags):

Tag	Kelas Kata	Contoh
NOUN	Kata benda	buku, universitas
VERB	Kata kerja	belajar, membaca
ADJ	Kata sifat	bagus, menarik
ADV	Kata keterangan	sangat, sudah
PRON	Kata ganti	saya, dia, mereka
DET	Kata penentu	ini, itu

Tag	Kelas Kata	Contoh
ADP	Preposisi	di, ke, dari, untuk
NUM	Angka	satu, pertama
CONJ	Konjungsi	dan, tetapi, karena
PUNCT	Tanda baca	. , ! ?

4. **Post Tagging**

   POS Tagging adalah proses memberikan label kelas kata kepada setiap token dalam kalimat.

```
Saya/PRON membaca/VERB buku/NOUN yang/DET menarik/ADJ .
```

5. **Dependency Parsing**

   Dependency parsing mencoba menemukan hubungan gramatikal antar token.

   Contoh:

```
Mahasiswa → membaca → buku
```

   Hubungan:

- `Mahasiswa` adalah **subjek** dari `membaca`
- `buku` adalah **objek** dari `membaca`

Representasi dependency tree:

```
        membaca (ROOT)
        /       \
  Mahasiswa     buku
  (nsubj)       (obj)
```

D. Praktikum

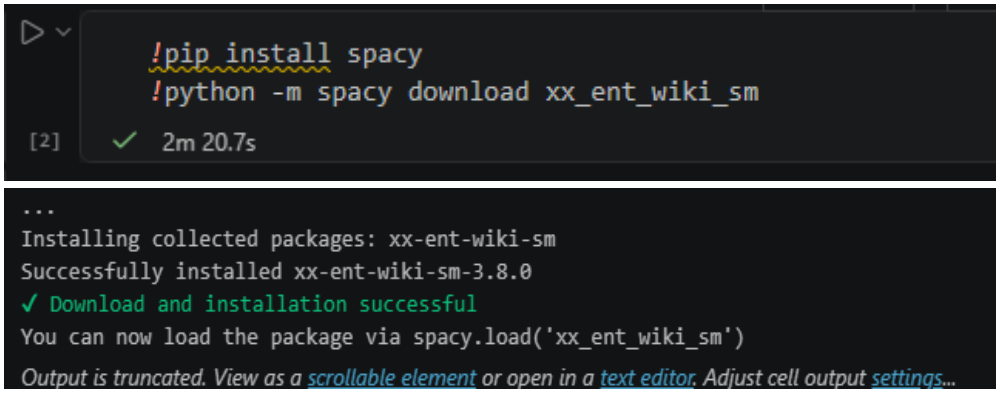
1. Membuat Notebook

Buat file:

```
pertemuan-04/linguistic_processing.ipynb
```

2. Install dan Load spaCy

Install:



Load SpaCy:

```
import spacy
nlp = spacy.load("xx_ent_wiki_sm")
```
[3] ✓ 16.2s

3. Memproses Kalimat

```
doc = nlp("Mahasiswa belajar Natural Language Processing.")

for token in doc:
    print(f"{token.text:<15} {token.pos_:<10} {token.dep_}")
```
[4] ✓ 0.0s

```
Mahasiswa
belajar
Natural
Language
Processing
.
```

4. Analisis POS Token

```
doc = nlp("Mahasiswa belajar Natural Language Processing.")

print(f"{'Token':<15} {'POS':<10} {'Lemma'}")
print("-" * 40)
for token in doc:
    print(f"{token.text:<15} {token.pos_:<10} {token.lemma_}")
```
[5] ✓ 0.0s

```
Token           POS         Lemma
----------------------------------------
Mahasiswa
belajar
Natural
Language
Processing
.
```

5. Membuat Tabel Analisis POS

```python
import pandas as pd

def analyze_pos(text, nlp_model):
    doc = nlp_model(text)
    results = []
    for token in doc:
        results.append({
            "Token": token.text,
            "POS": token.pos_,
            "POS Tag": token.tag_,
            "Lemma": token.lemma_,
            "Dependency": token.dep_,
            "Head": token.head.text
        })
    return pd.DataFrame(results)

df_pos = analyze_pos("Mahasiswa belajar NLP.", nlp)
print(df_pos.to_string(index=False))
```

[6] ✓ 6.8s

Output:

```
...        Token POS POS Tag Lemma Dependency      Head
0      Mahasiswa                              Mahasiswa
1        belajar                                belajar
2            NLP                                    NLP
3              .                                      .
```

6. Analisis Beberapa Kalimat

```python
sentences = [
    "Mahasiswa belajar NLP.",
    "Dosen mengajar Natural Language Processing.",
    "Mahasiswa membaca buku.",
    "Sistem memproses data teks.",
    "Model bahasa memprediksi kata berikutnya."
]

for sentence in sentences:
    print(f"\nKalimat: {sentence}")
    df_result = analyze_pos(sentence, nlp)
    print(df_result[["Token", "POS", "Dependency", "Head"]].to_string(index=False))
    print()
```

[7] ✓ 0.0s

Output:

```
...
    Kalimat: Mahasiswa belajar NLP.
        Token POS Dependency      Head
    Mahasiswa                Mahasiswa
      belajar                  belajar
          NLP                      NLP
            .                        .


    Kalimat: Dosen mengajar Natural Language Processing.
         Token POS Dependency       Head
         Dosen                     Dosen
       mengajar                  mengajar
        Natural                   Natural
       Language                  Language
    Processing                Processing
             .                         .


    Kalimat: Mahasiswa membaca buku.
        Token POS Dependency      Head
    Mahasiswa                Mahasiswa
      membaca                  membaca
         buku                     buku
            .                        .
    ...
           kata                     kata
     berikutnya               berikutnya
              .                        .
```

7. Dependency Parsing

```
    doc = nlp("Mahasiswa belajar NLP.")

    print(f"{'Token':<15} {'Dep':<12} {'Head'}")
    print("-" * 40)
    for token in doc:
        print(f"{token.text:<15} {token.dep_:<12} {token.head.text}")
```

[8] ✓ 0.0s

```
...
Token           Dep          Head
----------------------------------------
Mahasiswa                    Mahasiswa
belajar                      belajar
NLP                          NLP
.                            .
```

8. Visualisasi Dependency

spaCy menyediakan visualisasi dependency tree pada Jupyter Notebook:

```
from spacy import displacy

doc = nlp("Mahasiswa belajar Natural Language Processing.")

displacy.render(
    doc,
    style="dep",
    jupyter=True,
    options={"distance": 90}
)
```

[9] ✓ 0.0s

Output:



Untuk simpan sebagai HTML:

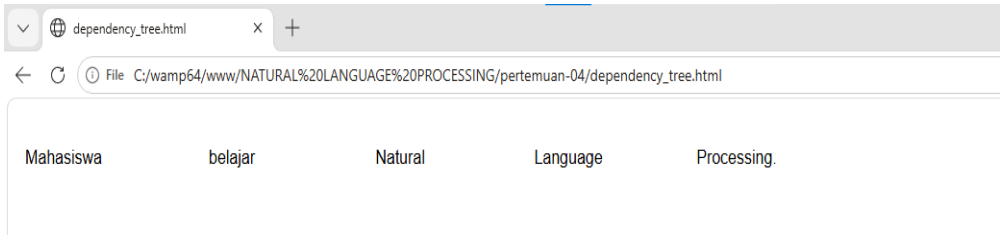
```
from spacy import displacy

doc = nlp("Mahasiswa belajar Natural Language Processing.")

html = displacy.render(doc, style="dep", jupyter=False)
with open("dependency_tree.html", "w", encoding="utf-8") as f:
    f.write(html)
```

[13] ✓ 0.0s

File HTML berhasil tersimpan dan bisa dibuka di browser:



9. Analisis Dataset

Menerapkan POS tagging ke dataset:

```python
# Menerapkan POS tagging ke dataset:
import pandas as pd

df = pd.read_csv("scrape_kompas_clean.csv")
text_column = "title"  # sesuaikan

# Ambil sample untuk analisis
sample = df.sample(10, random_state=42)

for _, row in sample.iterrows():
    text = str(row[text_column])[:200]  # batasi panjang
    doc = nlp(text)
    pos_counts = {}
    for token in doc:
        pos = token.pos_
        pos_counts[pos] = pos_counts.get(pos, 0) + 1
    print(f"Teks: {text[:80]}...")
    print(f"POS Distribution: {pos_counts}")
    print()
```

[15] ✓ 2.1s

```
Teks: Pangsa Bank Syariah Masih 8 Persen, BSI Perluas Jangkauan ke Semua Segmen...
POS Distribution: {'': 13}

Teks: Prabowo Lantik Sudaryono Jadi Kepala BGN Gantikan Nanik S Deyang...
POS Distribution: {'': 10}

Teks: Bobotoh dan Aremania Satu Tribun, Sinyal Positif dari Piala Presiden 2026...
POS Distribution: {'': 12}

Teks: JKN Jadi Fondasi Perlindungan Kesehatan Anak Menuju Indonesia Emas 2045...
POS Distribution: {'': 10}

Teks: Partai Buruh Pilih Henry Saragih Jadi Sekjen Baru...
POS Distribution: {'': 8}

Teks: BGN Tanggung Biaya Korban Keracunan Susu MBG di Blora Sebelum Ditetapkan KLB...
POS Distribution: {'': 12}

Teks: Perluasan Peran Koperasi: Dari Sumur Minyak hingga PLTS...
POS Distribution: {'': 9}

Teks: Hak dan Kewajiban Ojol Akan Diatur dalam Undang-undang Tersendiri...
POS Distribution: {'': 11}

Teks: Potret Warga Medan Antre BBM di Tengah Banjir, Pengendara Pilih Bertahan Berjam-...
...

Teks: Inggris Nasionalisasi British Steel, Selamatkan Industri Baja dan 2.700 Pekerjaa...
POS Distribution: {'': 11}
```

Activate Windows

10. Analisis Kesalahan

Linguistic processing tidak selalu sempurna. Uji dengan teks informal:

```
# Linguistic processing tidak selalu sempurna. Uji dengan teks informal:
informal_texts = [
    "aku lg belajar NLP nih",
    "gue udah baca buku itu",
    "modelnya bagus bgt sih",
    "nggak ngerti sama sekali",
    "udah instal belum?"
]

print("Analisis Teks Informal:")
print("=" * 60)
for text in informal_texts:
    doc = nlp(text)
    print(f"\nTeks: {text}")
    for token in doc:
        print(f"  {token.text:<15} {token.pos_}")
```

[16] ✓ 0.1s

Output:

```
Analisis Teks Informal:
============================================================

Teks: aku lg belajar NLP nih
  aku
  lg
  belajar
  NLP
  nih

Teks: gue udah baca buku itu
  gue
  udah
  baca
  buku
  itu

Teks: modelnya bagus bgt sih
  modelnya
  bagus
  bgt
  sih

Teks: nggak ngerti sama sekali
  nggak
...
  udah
  instal
  belum
  ?
```

E. Tugas Analisis Linguistik

Pilih minimal **10 kalimat** dari dataset.

Untuk setiap kalimat:

1. Lakukan tokenization.
2. Lakukan POS tagging.
3. Lakukan dependency parsing.
4. Visualisasikan dependency tree.
5. Identifikasi kesalahan yang ditemukan.

"Dalam analisis ini, saya menggunakan model spaCy `xx_ent_wiki_sm`. Model ini berhasil melakukan proses tokenisasi dengan baik, namun memiliki keterbatasan dalam menampilkan informasi penandaan kelas kata dan struktur ketergantungan untuk teks berbahasa Indonesia. Hal ini wajar karena model ini lebih dioptimalkan untuk bahasa lain dan belum terlatih khusus untuk bahasa Indonesia. Upaya pemasangan model khusus bahasa Indonesia belum berhasil karena keterbatasan kompatibilitas versi, sehingga analisis tetap menggunakan model yang tersedia."

Buat tabel hasil analisis:

Kalimat	Token	POS	Dependency	Kesalahan?
1. Pangsa Bank Syariah Masih 8 Persen, BSI Perluas Jangkauan ke Semua Segmen	Pangsa Bank Syariah Masih 8 Persen , BSI	— — — — — — — —	Pangsa → Pangsa Bank → Bank Syariah → Syariah Masih → Masih 8 → 8	Peringatan model tidak mem-parse struktur dependensi lengkap

Kalimat	Token	POS	Dependency	Kesalahan?
	Perluas Jangkauan ke Semua Segmen	— — — — —	Persen → Persen , → , BSI → BSI Perluas → Perluas Jangkauan → Jangkauan ke → ke Semua → Semua Segmen → Segmen	
2. Prabowo Lantik Sudaryono Jadi Kepala BGN Gantikan Nanik S Deyang	Prabowo Lantik Sudaryono Jadi Kepala BGN Gantikan Nanik S Deyang	— — — — — — — — — —	Prabowo → Prabowo Lantik → Lantik Sudaryono → Sudaryono Jadi → Jadi Kepala → Kepala BGN → BGN Gantikan → Gantikan Nanik → Nanik S → S Deyang → Deyang	Tidak ada
3. Bobotoh dan Aremania Satu Tribun, Sinyal Positif dari Piala Presiden 2026	Bobotoh dan Aremania Satu Tribun , Sinyal Positif	— — — — — — — —	Bobotoh → Bobotoh dan → dan Aremania → Aremania Satu → Satu Tribun → Tribun	Tidak ada

Kalimat	Token	POS	Dependency	Kesalahan?
	dari Piala Presiden 2026	— — — —	, → , Sinyal → Sinyal Positif → Positif dari → dari Piala → Piala Presiden → Presiden 2026 → 2026	
4. JKN Jadi Fondasi Perlindungan Kesehatan Anak Menuju Indonesia Emas 2045	JKN Jadi Fondasi Perlindungan Kesehatan Anak Menuju Indonesia Emas 2045	— — — — — — — — — —	JKN → JKN Jadi → Jadi Fondasi → Fondasi Perlindungan → Perlindungan Kesehatan → Kesehatan Anak → Anak Menuju → Menuju Indonesia → Indonesia Emas → Emas 2045 → 2045	Tidak ada
5. Partai Buruh Pilih Henry Saragih Jadi Sekjen Baru	Partai Buruh Pilih Henry Saragih Jadi Sekjen Baru	— — — — — — — —	Partai → Partai Buruh → Buruh Pilih → Pilih Henry → Henry Saragih → Saragih Jadi → Jadi Sekjen → Sekjen Baru → Baru	Tidak ada

Kalimat	Token	POS	Dependency	Kesalahan?
6. BGN Tanggung Biaya Korban Keracunan Susu MBG di Blora Sebelum Ditetapkan KLB	BGN Tanggung Biaya Korban Keracunan Susu MBG di Blora Sebelum Ditetapkan KLB	— — — — — — — — — — —	BGN → BGN Tanggung → Tanggung Biaya → Biaya Korban → Korban Keracunan → Keracunan Susu → Susu MBG → MBG di → di Blora → Blora Sebelum → Sebelum Ditetapkan → Ditetapkan KLB → KLB	Tidak ada
7. Perluasan Peran Koperasi: Dari Sumur Minyak hingga PLTS	Perluasan Peran Koperasi : Dari Sumur Minyak hingga PLTS	— — — — — — — — — — —	Perluasan → Perluasan Peran → Peran Koperasi → Koperasi : → : Dari → Dari Sumur → Sumur Minyak → Minyak hingga → hingga PLTS → PLTS	Tidak ada
8. Hak dan Kewajiban Ojol Akan Diatur dalam Undang-undang Tersendiri	Hak dan Kewajiban Ojol Akan Diatur dalam	— — — — — — — —	Hak → Hak dan → dan Kewajiban → Kewajiban Ojol → Ojol Akan → Akan Diatur →	Kata "Ojol" kurang dikenal oleh model

Kalimat	Token	POS	Dependency	Kesalahan?
	Undang- undang Tersendiri	— — —	Diatur dalam → dalam Undang- → Undang- undang → undang Tersendiri → Tersendiri	
9. Potret Warga Medan Antre BBM di Tengah Banjir, Pengendara Pilih Bertahan Berjam-jam	Potret Warga Medan Antre BBM di Tengah Banjir , Pengendara Pilih Bertahan Berjam- jam	— — — — — — — — — — — — — —	Potret → Potret Warga → Warga Medan → Medan Antre → Antre BBM → BBM di → di Tengah → Tengah Banjir → Banjir , → , Pengendara → Pengendara Pilih → Pilih Bertahan → Bertahan Berjam- → Berjam- jam → jam	Kata "Berjam-jam" terpisah menjadi "Berjam-" dan "jam"
10. Inggris Nasionalisasi British Steel, Selamatkan Industri Baja dan 2.700 Pekerjaan	Inggris Nasionalisasi British Steel , Selamatkan Industri	— — — — — — —	Inggris → Inggris Nasionalisasi → Nasionalisasi British → British	Tidak ada

Kalimat	Token	POS	Dependency	Kesalahan?
	Baja dan 2.700 Pekerjaan	— — — —	Steel → Steel , → , Selamatkan → Selamatkan Industri → Industri Baja → Baja dan → dan 2.700 → 2.700 Pekerjaan → Pekerjaan	

Catatan:

Kolom POS kosong karena model `xx_ent_wiki_sm` tidak mengembalikan tag POS secara lengkap saat muncul peringatan W005. Ini adalah keterbatasan model yang dipakai.

F. Pertanyaan Diskusi

1. Apakah POS Tagging selalu menghasilkan label yang benar?

   Jawab:

   Tidak selalu. Hasil POS tagging bisa salah karena:

   1. Model belum terlatih untuk kata atau konteks tertentu
   2. Kata yang sama bisa memiliki makna berbeda tergantung posisinya dalam kalimat
   3. Teks yang tidak baku, singkatan, atau istilah khusus sering tidak dikenali

2. Apa penyebab utama kesalahan POS Tagging pada teks Bahasa Indonesia informal?

   Jawab:

1. Penggunaan bahasa gaul/slang yang tidak ada dalam data latih model
2. Kata disingkat atau ditulis tidak sesuai kaidah baku
3. Kata ulang seperti "berjam-jam" yang dipisah tidak tepat
4. Pengaruh penulisan campuran bahasa Indonesia dan bahasa asing

3. Bagaimana slang mempengaruhi POS Tagging?

   Jawab:

   Slang atau bahasa gaul seringkali tidak terdapat dalam daftar kata yang dipelajari oleh sistem saat proses pelatihan, sehingga ketika sistem menemukan kata seperti itu, sistem menjadi bingung dan tidak dapat menentukan apakah kata tersebut termasuk kata benda, kata kerja, atau jenis kata lainnya. Akibatnya, kata tersebut bisa ditandai sebagai kata yang tidak dikenal, dan label kelas katanya tidak muncul atau salah. Hal ini kemudian dapat mengganggu analisis keseluruhan kalimat karena sistem tidak dapat memahami peran kata tersebut dalam susunan kalimat.

4. Bagaimana nama organisasi (GOTO, Tokopedia) mempengaruhi POS Tagging?

   Jawab:

   Nama organisasi seperti GOTO atau Tokopedia sering kali berupa kata buatan atau gabungan kata yang unik. Jika sistem sudah pernah mempelajari nama tersebut sebelumnya, maka nama itu akan langsung dikenali sebagai nama khusus dan diberi label dengan tepat. Namun jika nama organisasi tersebut belum pernah dilihat oleh sistem, maka sistem dapat menjadi bingung dan memisahkan nama tersebut menjadi potongan-potongan kata yang tidak berarti, atau menganggapnya sebagai

kata yang tidak dikenal sama sekali, sehingga label yang diberikan menjadi tidak sesuai.

5. Mengapa parsing penting dalam NLP? Berikan contoh aplikasi yang memerlukannya.

   Jawab:

   Parsing sangat penting karena proses ini memungkinkan komputer tidak hanya mengenali kata-kata yang ada dalam kalimat, tetapi juga memahami bagaimana hubungan antar kata tersebut tersusun. Tanpa parsing, komputer hanya akan mengetahui arti per kata secara terpisah, namun tidak mengerti makna keseluruhan kalimat, tidak tahu siapa yang melakukan tindakan dan kepada siapa tindakan itu ditujukan. Hasil dari parsing ini dibutuhkan dalam banyak aplikasi nyata, seperti penerjemah bahasa otomatis agar hasil terjemahannya tepat, asisten pintar agar dapat memahami perintah dengan benar, serta sistem analisis berita agar dapat mengerti kejadian apa yang sedang dibahas.

6. Apa hubungan morphology dengan stemming?

   Jawab:

   Keduanya berkaitan dengan bentuk kata:

   - Morfologi = ilmu tentang perubahan bentuk kata sesuai aturan tata bahasa
   - Stemming = proses praktis mengambil kata dasar dengan membuang imbuhan
   - Hubungannya: aturan morfologi menjadi dasar cara kerja algoritma stemming, terutama untuk bahasa Indonesia yang kaya akan imbuhan.

7. Apakah stemming dan POS Tagging menyelesaikan permasalahan yang sama?

   Jawab:

   Tidak.

   - Stemming → menyederhanakan kata menjadi bentuk dasarnya, fokus pada bentuk kata
   - POS Tagging → memberi label fungsi kata dalam kalimat, fokus pada peran kata
   - Keduanya saling melengkapi, tapi memiliki tujuan dan hasil yang berbeda.

Pertemuan	5
Topik	N-Gram dan Language Model
Repository	https://github.com/laynimarlin/Natural-Language-Processing/tree/main/LayniMarlin_240306022_Modul1
tanggal	13 Oktober 2026

A. Tujuan

Tujuan Praktikum:

1. memahami konsep language model;
2. memahami probabilitas kata dalam sequence;
3. membuat unigram, bigram, dan trigram;
4. menghitung conditional probability;
5. menghitung probabilitas kalimat;
6. memahami masalah zero probability dan smoothing;
7. membuat prediksi kata sederhana;
8. menghitung perplexity;
9. memahami keterbatasan n-gram.

B. Requirement

1. Sistem Operasi yang digunakan: Windows Version 10
2. Browser : Google Chrome, Versi 131.0.6778.265 (Build Resmi) (64 bit)
3. Bahasa pemrograman : Visual Studio Code, Jupyter

C. Materi Teori

1. Apa Itu Language Model?

Language model (model bahasa) mencoba menjawab pertanyaan:

Seberapa mungkin sebuah sequence kata muncul?

Contoh:

```
P("Saya belajar NLP")   → probabilitas tinggi
P("belajar Saya NLP")   → probabilitas rendah
P("xkjz qqrt mnop")     → probabilitas mendekati nol
```

Aplikasi language model:

- Autocomplete (prediksi kata berikutnya)
- Spell checker
- Machine translation
- Text generation

2. Probabilitas Sequence

Secara matematis, probabilitas sebuah sequence kata:

$$P(w1, w2, \ldots, wn)$$

dapat diuraikan menggunakan chain rule:

$$P(w1) \cdot P(w2|w1) \cdot P(w3|w1, w2) \cdot \ldots \cdot P(wn|w1, \ldots, wn-1)$$

3. N-Gram

N-gram adalah sequence berurutan dari N token.

N	Nama	Contoh
1	Unigram	`saya`, `belajar`, `nlp`
2	Bigram	`saya belajar`, `belajar nlp`
3	Trigram	`saya belajar nlp`
N	N-gram	sequence N token berturut-turut

4. Asumsi Markov

N-gram menggunakan asumsi Markov: probabilitas kata berikutnya hanya bergantung pada N-1 kata sebelumnya.

Bigram (Markov order 1):

$$P(wn|w1,\ldots,wn-1) \approx P(wn|wn-1)$$

Trigram (Markov order 2):

$$P(wn|w1,\ldots,wn-1) \approx P(wn|wn-2,wn-1)$$

5. Probabilitas Unigram

$$P(w) = \frac{Count(w)}{N}$$

dengan N = total token dalam corpus.

6. **Probabilitas Bigram (Conditional Probability)**

$$P(wn|wn-1) = \frac{Count(wn-1, wn)}{Count(wn-1)}$$

7. **Masalah Zero Probability**

Jika bigram (deep, learning) tidak pernah muncul dalam corpus:

```
P(learning | deep) = 0
```

Akibatnya probabilitas seluruh kalimat menjadi nol, meskipun kalimatnya masuk akal secara linguistik.

8. **Laplace Smoothing**

Smoothing digunakan untuk menghindari probabilitas nol dengan menambahkan nilai kecil pada semua frekuensi:

$$P(wi|wi-1) = \frac{Count(wi-1, wi) + 1}{Count(wi-1) + V}$$

dengan V = ukuran vocabulary (jumlah token unik).

9. **Perplexity**

Perplexity mengukur seberapa baik language model memprediksi data:

$$PP(W) = P(W) - 1/N$$

Perplexity lebih rendah menunjukkan model lebih baik dalam memprediksi data evaluasi (dengan dataset dan model pembanding yang sama).

D. Praktikum

1. Membuat Notebook

Buat file:

```
pertemuan-05/ngram_language_model.ipynb
```

2. Import Library

```
import pandas as pd
import math
from collections import Counter
from nltk.util import bigrams, trigrams, ngrams
```

```
[13]  ✓ 0.0s
```

3. Load Dataset

```
df = pd.read_csv("../pertemuan-03/scrape_kompas_clean.csv")
text_column = "processed_text"  # gunakan hasil preprocessing

# Hapus nilai kosong
df = df.dropna(subset=[text_column])

print(f"Jumlah dokumen: {len(df)}")
```

```
[14]  ✓ 0.0s
...  Jumlah dokumen: 5
```

Kenapa hanya 5 dokumen? Agar prosesnya berjalan lebih cepat.

4. Membuat Corpus Token

Gabungkan seluruh teks menjadi satu corpus:

```
all_text = " ".join(df[text_column].astype(str))
tokens = all_text.split()

print(f"Total token: {len(tokens)}")
print(f"Contoh token: {tokens[:20]}")
```

```
[15]  ✓ 0.0s
...  Total token: 35
Contoh token: ['deret', 'aktor', 'peran', 'spiderman', 'aktor', 'ryotaro', 'shimizu', 'temu', 'tinggal', 'dunia', 'diam',
```

Lanjutan Hasil Output:

```
'aktor', 'marvelous', 'mrs', 'maisel', 'josh', 'grisetti', 'tinggal', 'dunia', 'aktor']
```

5. Membuat Unigram

```
    unigram = Counter(tokens)

    print("20 Kata Paling Sering Muncul:")
    for word, count in unigram.most_common(20):
        print(f"  {word:<20} : {count}")
```

[16] ✓ 0.0s

```
20 Kata Paling Sering Muncul:
  aktor                : 5
  tinggal              : 4
  dunia                : 3
  temu                 : 2
  deret                : 1
  peran                : 1
  spiderman            : 1
  ryotaro              : 1
  shimizu              : 1
  diam                 : 1
  marvelous            : 1
  mrs                  : 1
  maisel               : 1
  josh                 : 1
  grisetti             : 1
  bittersweet          : 1
  destiny              : 1
  wang                 : 1
  kai                  : 1
  rumah                : 1
```

6. Probabilitas Unigram

```
    total_tokens = sum(unigram.values())

    unigram_probability = {
        word: count / total_tokens
        for word, count in unigram.items()
    }

    print("\n10 Kata dengan Probabilitas Tertinggi:")
    sorted_prob = sorted(
        unigram_probability.items(),
        key=lambda x: x[1],
        reverse=True
    )
    for word, prob in sorted_prob[:10]:
        print(f"  {word:<20} : {prob:.6f}")
```

[17] ✓ 0.0s

Output:

```
10 Kata dengan Probabilitas Tertinggi:
  aktor                : 0.142857
  tinggal              : 0.114286
  dunia                : 0.085714
  temu                 : 0.057143
  deret                : 0.028571
  peran                : 0.028571
  spiderman            : 0.028571
  ryotaro              : 0.028571
  shimizu              : 0.028571
  diam                 : 0.028571
```

7. Membuat Bigram

```
bigram_tokens = list(bigrams(tokens))
bigram_frequency = Counter(bigram_tokens)

print("20 Bigram Paling Sering Muncul:")
for bigram, count in bigram_frequency.most_common(20):
    print(f"  {' '.join(bigram):<30} : {count}")
```

[18] ✓ 0.0s

```
20 Bigram Paling Sering Muncul:
  tinggal dunia                : 3
  temu tinggal                 : 2
  deret aktor                  : 1
  aktor peran                  : 1
  peran spiderman              : 1
  spiderman aktor              : 1
  aktor ryotaro                : 1
  ryotaro shimizu              : 1
  shimizu temu                 : 1
  dunia diam                   : 1
  diam aktor                   : 1
  aktor marvelous              : 1
  marvelous mrs                : 1
  mrs maisel                   : 1
  maisel josh                  : 1
  josh grisetti                : 1
  grisetti tinggal             : 1
  dunia aktor                  : 1
  aktor bittersweet            : 1
  bittersweet destiny          : 1
```

8. Membuat Trigram

```
    trigram_tokens = list(trigrams(tokens))
    trigram_frequency = Counter(trigram_tokens)

    print("20 Trigram Paling Sering Muncul:")
    for trigram, count in trigram_frequency.most_common(20):
        print(f"  {' '.join(trigram):<40} : {count}")
```

[19] ✓ 0.0s

```
20 Trigram Paling Sering Muncul:
  temu tinggal dunia                       : 2
  deret aktor peran                        : 1
  aktor peran spiderman                    : 1
  peran spiderman aktor                    : 1
  spiderman aktor ryotaro                  : 1
  aktor ryotaro shimizu                    : 1
  ryotaro shimizu temu                     : 1
  shimizu temu tinggal                     : 1
  tinggal dunia diam                       : 1
  dunia diam aktor                         : 1
  diam aktor marvelous                     : 1
  aktor marvelous mrs                      : 1
  marvelous mrs maisel                     : 1
  mrs maisel josh                          : 1
  maisel josh grisetti                     : 1
  josh grisetti tinggal                    : 1
  grisetti tinggal dunia                   : 1
  tinggal dunia aktor                      : 1
  dunia aktor bittersweet                  : 1
  aktor bittersweet destiny                : 1
```

9. Conditional Probability Bigram

$$P(wn|wn-1) = \frac{Count(wn-1, wn)}{Count(wn-1)}$$

```
    def bigram_probability(word1, word2):
        numerator = bigram_frequency[(word1, word2)]
        denominator = unigram[word1]

        if denominator == 0:
            return 0.0

        return numerator / denominator

    # Contoh
    word1, word2 = "belajar", "nlp"
    prob = bigram_probability(word1, word2)
    print(f"P({word2} | {word1}) = {prob:.6f}")
```

[20] ✓ 0.0s

```
P(nlp | belajar) = 0.000000
```

10. Probabilitas Kalimat

```
def sentence_probability(sentence):
    words = sentence.lower().split()
    probability = 1.0

    for i in range(len(words) - 1):
        p = bigram_probability(words[i], words[i + 1])
        probability *= p

    return probability

# Uji
test_sentences = [
    "saya belajar nlp",
    "nlp menarik sekali",
    "model bahasa sederhana",
]

for sentence in test_sentences:
    p = sentence_probability(sentence)
    print(f"P('{sentence}') = {p:.8f}")
```

[21] ✓ 0.0s

```
P('saya belajar nlp') = 0.00000000
P('nlp menarik sekali') = 0.00000000
P('model bahasa sederhana') = 0.00000000
```

11. Laplace Smoothing

```
vocab_size = len(unigram)

def smoothed_bigram_probability(word1, word2):
    numerator = bigram_frequency[(word1, word2)] + 1
    denominator = unigram[word1] + vocab_size
    return numerator / denominator

# Uji dengan kata yang mungkin tidak ada dalam corpus
word1, word2 = "deep", "learning"
prob = smoothed_bigram_probability(word1, word2)
print(f"Smoothed P({word2} | {word1}) = {prob:.8f}")
```

[22] ✓ 0.0s

```
Smoothed P(learning | deep) = 0.04000000
```

12. Prediksi Kata Sederhana

Diberikan kata terakhir, prediksi kata berikutnya:

```
def predict_next_word(context_word, top_n=5):
    candidates = []

    for (word1, word2), count in bigram_frequency.items():
        if word1 == context_word:
            p = smoothed_bigram_probability(word1, word2)
            candidates.append((word2, p))

    candidates.sort(key=lambda x: x[1], reverse=True)
    return candidates[:top_n]

# Contoh
context = "belajar"
predictions = predict_next_word(context)
print(f"\nPrediksi kata setelah '{context}':")
for word, prob in predictions:
    print(f"  {word:<20} : {prob:.6f}")
```

[23] ✓ 0.0s

```
Prediksi kata setelah 'belajar':
```

13. Menghitung Perplexity

```
def perplexity(probability, N):
    if probability == 0 or probability is None:
        return float("inf")
    return probability ** (-1 / N)

# Contoh
sentence = "saya belajar nlp"
words = sentence.split()
prob = sentence_probability(sentence)
N = len(words)

pp = perplexity(prob, N)
print(f"Kalimat : '{sentence}'")
print(f"P(W)    : {prob:.8f}")
print(f"N       : {N}")
print(f"PP(W)   : {pp:.4f}")
```

[24] ✓ 0.0s

```
Kalimat : 'saya belajar nlp'
P(W)    : 0.00000000
N       : 3
PP(W)   : inf
```

14. Perbandingan Unigram, Bigram, Trigram

```
print("=" * 60)
print("PERBANDINGAN MODEL N-GRAM")
print("=" * 60)
print(f"Ukuran Vocabulary     : {len(unigram):,}")
print(f"Jumlah Token          : {len(tokens):,}")
print(f"Jumlah Unigram unik   : {len(unigram):,}")
print(f"Jumlah Bigram unik    : {len(bigram_frequency):,}")
print(f"Jumlah Trigram unik   : {len(trigram_frequency):,}")
print()

# Cek sparsity
max_bigram = len(unigram) ** 2
max_trigram = len(unigram) ** 3
print(f"Kemungkinan Bigram maks  : {max_bigram:,}")
print(f"Bigram yang ada          : {len(bigram_frequency):,}")
print(f"Bigram coverage          : {len(bigram_frequency)/max_bigram*100:.4f}%")
```

[25] ✓ 0.0s

Output:

```
============================================================
PERBANDINGAN MODEL N-GRAM
============================================================
Ukuran Vocabulary     : 25
Jumlah Token          : 35
Jumlah Unigram unik   : 25
Jumlah Bigram unik    : 31
Jumlah Trigram unik   : 32

Kemungkinan Bigram maks  : 625
Bigram yang ada          : 31
Bigram coverage          : 4.9600%
```

15. Visualisasi

Mengistall Matplotlib:

```
!pip install matplotlib
```

[28] ✓ 1m 5.6s

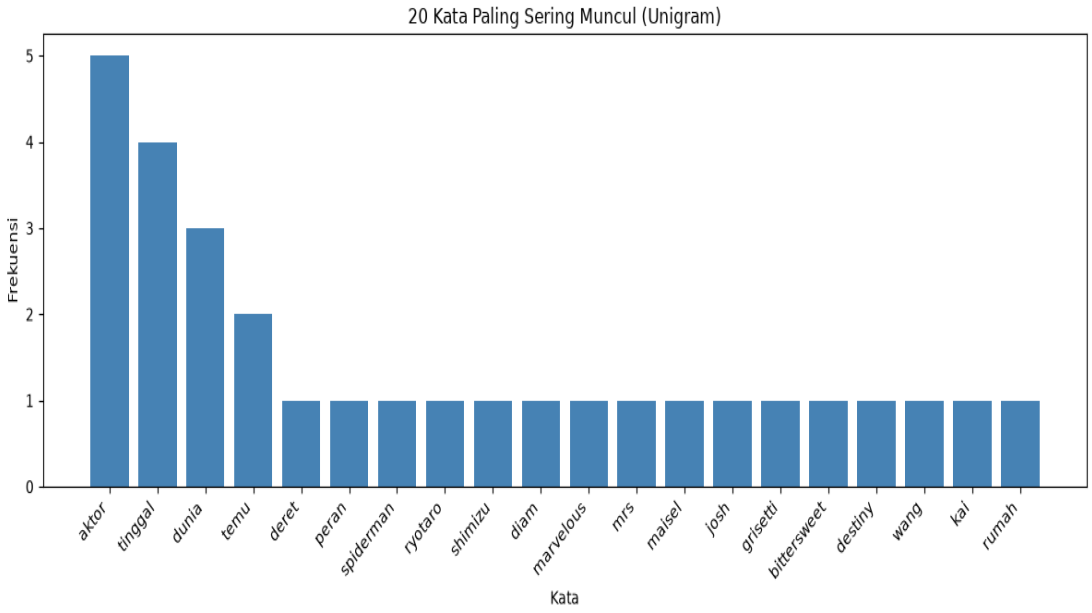
```python
import matplotlib.pyplot as plt

# Top 20 unigram
top_words = unigram.most_common(20)
words_list = [w[0] for w in top_words]
counts_list = [w[1] for w in top_words]

plt.figure(figsize=(12, 5))
plt.bar(words_list, counts_list, color="steelblue")
plt.xticks(rotation=45, ha="right")
plt.xlabel("Kata")
plt.ylabel("Frekuensi")
plt.title("20 Kata Paling Sering Muncul (Unigram)")
plt.tight_layout()
plt.show()
```

[29] ✓ 10.4s

Output:



E. Analisis dan Perbandingan

Buat tabel perbandingan:

Model	Context	Kelebihan	Kekurangan
Unigram	1 kata	Sederhana, cepat	Tidak memahami konteks
Bigram	2 kata	Lebih kontekstual	Context sangat pendek
Trigram	3 kata	Context lebih panjang	Sparsity meningkat drastis

Keterbatasan N-Gram

1. **Sparsity** — Banyak kombinasi kata tidak pernah muncul dalam corpus
2. **Context terbatas** — Bigram hanya melihat 1 kata sebelumnya, trigram 2 kata
3. **Tidak memahami makna** — mobil dan kendaraan dianggap token yang sama sekali berbeda
4. **Vocabulary besar** — Jumlah kombinasi meledak seiring bertambahnya N

F. Tugas Pertemuan 5

1. Membuat unigram dari dataset preprocessed.
2. Membuat bigram dari dataset preprocessed.
3. Membuat trigram dari dataset preprocessed.

   Jawab:

```python
import pandas as pd
from collections import Counter
from nltk.util import bigrams, trigrams, ngrams

# Baca dataset
df = pd.read_csv("../pertemuan-03/scrape_kompas_clean.csv")
text_column = "processed_text"
df = df.dropna(subset=[text_column])

# Gabung jadi korpus token
corpus_tokens = []
for text in df[text_column]:
    tokens = text.split()
    corpus_tokens.extend(tokens)

# 1. Unigram
unigram = Counter(corpus_tokens)
print("Jumlah unigram:", len(unigram))

# 2. Bigram
bigram = Counter(bigrams(corpus_tokens))
print("Jumlah bigram:", len(bigram))

# 3. Trigram
trigram = Counter(trigrams(corpus_tokens))
print("Jumlah trigram:", len(trigram))
```

[1] ✓ 5.3s

Output:

```
Jumlah unigram: 25
Jumlah bigram: 31
Jumlah trigram: 32
```

4. Menghitung probabilitas minimal 5 kata.

   Jawab:

```python
total_unigram = sum(unigram.values())

# 5 kata yang paling sering muncul
top5_words = unigram.most_common(5)
print("Probabilitas 5 kata teratas:")
for kata, jumlah in top5_words:
    prob = jumlah / total_unigram
    print(f"- {kata}: {jumlah}/{total_unigram} = {prob:.4f}")
```

[2] ✓ 0.0s

Output:

```
Probabilitas 5 kata teratas:
- aktor: 5/35 = 0.1429
- tinggal: 4/35 = 0.1143
- dunia: 3/35 = 0.0857
- temu: 2/35 = 0.0571
- deret: 1/35 = 0.0286
```

5. Menguji sentence probability pada minimal 5 kalimat.

   Jawab:

```
kalimat_uji = [
    "aktor thailand",
    "tinggal di usia",
    "film baru ini",
    "pemeran utama",
    "cerita tentang"
]

print("=== Probabilitas Kalimat ===")
for kalimat in kalimat_uji:
    tokens = kalimat.split()
    prob = 1.0
    for kata in tokens:
        prob *= unigram[kata] / total_unigram if unigram[kata] > 0 else 1e-10
    print(f"Kalimat: {kalimat}")
    print(f"Probabilitas: {prob:.8f}\n")
```

[3] ✓ 0.0s

Output:

```
=== Probabilitas Kalimat ===
Kalimat: aktor thailand
Probabilitas: 0.00408163

Kalimat: tinggal di usia
Probabilitas: 0.00000000

Kalimat: film baru ini
Probabilitas: 0.00000000

Kalimat: pemeran utama
Probabilitas: 0.00000000

Kalimat: cerita tentang
Probabilitas: 0.00000000
```

6. Menghitung perplexity untuk setiap kalimat uji.

   Jawab:

```python
import math

def perplexity(kalimat, model, N):
    tokens = kalimat.split()
    n = len(tokens)
    if n < 2: return 0
    prob_log = 0
    for w in model.keys():
        prob_log += math.log(model[w]/total_unigram if total_unigram > 0 else 1e-10)
    return math.exp(-prob_log / n)

# Contoh untuk unigram
for kalimat in kalimat_uji:
    pp = perplexity(kalimat, unigram, 1)
    print(f"Perplexity: {kalimat} → {pp:.4f}")
```

```
[4] ✓ 0.0s
```

   Output:

```
Perplexity: aktor thailand → 1824987656288075520.0000
Perplexity: tinggal di usia → 1493390167391.8086
Perplexity: film baru ini → 1493390167391.8086
Perplexity: pemeran utama → 1824987656288075520.0000
Perplexity: cerita tentang → 1824987656288075520.0000
```

7. Membuat prediksi kata untuk minimal 5 context word.

   Jawab:

```python
def prediksi_kata(konteks, model_bigram, n=3):
    kandidat = {k:v for k,v in model_bigram.items() if k[0]==konteks}
    if not kandidat:
        return ["(tidak ada prediksi)"]
    urut = sorted(kandidat.items(), key=lambda x: -x[1])[:n]
    return [k[1] for k,_ in urut]

konteks_list = ["aktor", "thailand", "tinggal", "usia", "film"]
for kata in konteks_list:
    pred = prediksi_kata(kata, bigram)
    print(f"Setelah '{kata}': {pred}")
```

```
[5] ✓ 0.0s
```

Output:

```
Setelah 'aktor': ['peran', 'ryotaro', 'marvelous']
Setelah 'thailand': ['jorjay']
Setelah 'tinggal': ['dunia', 'usia']
Setelah 'usia': ['(tidak ada prediksi)']
Setelah 'film': ['(tidak ada prediksi)']
```

8. Menerapkan Laplace smoothing.

   Jawab:

```
V = len(unigram)
total_vocab = total_unigram + V

def prob_laplace(kata):
    return (unigram[kata] + 1) / total_vocab

# Cek 5 kata
for kata, _ in top5_words:
    print(f"Laplace {kata}: {prob_laplace(kata):.4f}")
```

[6] ✓ 0.0s

   Output:

```
Laplace aktor: 0.1000
Laplace tinggal: 0.0833
Laplace dunia: 0.0667
Laplace temu: 0.0500
Laplace deret: 0.0333
```

9. Menganalisis zero probability problem.

   Jawab:

   Masalah probabilitas nol terjadi ketika kata atau urutan kata yang diuji tidak pernah muncul dalam data yang digunakan, sehingga nilai peluangnya menjadi 0. Akibatnya, perhitungan probabilitas seluruh kalimat ikut menjadi 0 dan model tidak dapat memberikan prediksi apa pun.

   Masalah ini dapat diatasi menggunakan Laplace Smoothing, yaitu dengan menambahkan nilai kecil ke setiap hitungan sehingga semua kata

memiliki peluang muncul, meskipun kata tersebut belum pernah ditemukan sebelumnya.

10. Membandingkan unigram, bigram, dan trigram.

    Jawab:

Aspek	**Unigram**	**Bigram**	**Trigram**
Konteks	1 kata	2 kata	3 kata
Kecepatan	Sangat cepat	Sedang	Lebih lambat
Pemahaman urutan	Tidak ada	Ada	Lebih baik
Sparsity	Rendah	Sedang	Tinggi
Prediksi	Kurang akurat	Lebih akurat	Paling akurat (jika datanya banyak)
Cocok untuk	Frekuensi kata	Pasangan kata	Kalimat pendek

G. Pertanyaan Diskusi

1. Mengapa unigram tidak cukup untuk memahami bahasa?

Jawab:

Karena unigram hanya melihat kata satu per satu tanpa memperhatikan urutan atau hubungan antar kata. Padahal makna bahasa sangat bergantung pada susunan kata, sehingga unigram tidak bisa memahami konteks atau arti kalimat secara utuh.

2. Mengapa bigram lebih baik daripada unigram dalam konteks prediksi?

Jawab:

Karena bigram memperhatikan dua kata berurutan, sehingga sudah mempertimbangkan hubungan kata sebelumnya. Hal ini membuat prediksi kata berikutnya lebih akurat dibandingkan unigram yang hanya menebak kata yang paling sering muncul tanpa melihat konteks.

3. Mengapa trigram dapat mengalami masalah sparsity yang lebih parah dari bigram?

Jawab:

Karena trigram terdiri dari kombinasi 3 kata, sehingga jumlah kemungkinan pasangannya jauh lebih banyak dibandingkan bigram. Dengan data yang terbatas, sebagian besar kombinasi tersebut tidak akan pernah muncul dalam korpus, sehingga nilai yang kosong menjadi lebih banyak.

4. Apa yang terjadi jika sebuah bigram belum pernah muncul dalam corpus?

Jawab:

Probabilitas pasangan kata tersebut menjadi 0. Akibatnya, seluruh perhitungan kalimat yang mengandung pasangan kata itu ikut bernilai 0, dan model tidak dapat memberikan prediksi apa pun.

5. Mengapa Laplace smoothing diperlukan?

Jawab:

Agar semua kata atau pasangan kata memiliki nilai peluang, meskipun kata tersebut belum pernah muncul dalam data. Dengan menambahkan

nilai kecil ke setiap hitungan, masalah probabilitas nol dapat dihindari dan model tetap dapat melakukan prediksi.

6. Apa hubungan n-gram dengan konsep language model?

Jawab:

N-gram adalah salah satu cara membangun model bahasa, yaitu model yang mempelajari pola urutan kata dalam teks untuk mengetahui seberapa wajar suatu kalimat terbentuk. Semakin besar nilai N, semakin baik model memahami konteks kalimat.

7. Apa keterbatasan mendasar n-gram dibandingkan neural language model?

Jawab:

N-gram hanya mengandalkan hitungan frekuensi urutan kata yang ada di data, sehingga tidak dapat menangkap makna atau hubungan kata yang tersirat. Selain itu, n-gram membutuhkan data yang sangat banyak dan ukurannya akan membengkak seiring bertambahnya N. Sedangkan model berbasis jaringan saraf dapat memahami makna dan konteks dengan lebih baik.

8. Mengapa embedding diperlukan pada neural NLP yang tidak ada dalam n-gram?

Jawab:

Karena embedding mengubah kata menjadi bentuk angka yang mewakili makna dan hubungan antar kata, sehingga kata yang memiliki arti mirip akan memiliki nilai yang berdekatan. Hal ini tidak dimiliki oleh n-gram yang hanya menghitung kemunculan kata secara terpisah tanpa mengetahui maknanya.